

Réseaux de neurones profonds

Une brève introduction

Résumé

Ce document constitue une introduction aux réseaux de neurones et aux éléments mathématiques qui permettent leur utilisation. Loin d'être exhaustif il aborde et illustre les notions d'apprentissage automatique, de descente de gradient et de descente de gradient stochastique.

Table des matières

1	Introduction	4
1.1	Apprentissage automatique et usages	4
1.1.1	Définitions et motivations	4
1.1.2	Modélisation et approches centrées sur les données	4
1.2	Notion de risque	6
1.2.1	Définition de la tâche et de la fonction de coût	6
1.2.2	Définition et différences entre risque réel et empirique	7
1.2.3	Notion de sur-apprentissage	8
1.2.4	Peut-on éviter le sur-apprentissage?	10
1.2.5	Notion de manifold	11
1.2.6	Conclusion	12
2	Neurones artificielles	15
2.1	À l'origine était le perceptron	15
2.1.1	Modèle du perceptron	15
2.1.2	Fonction d'activation et limitations	16
2.1.3	Non-linéarités	18
2.1.4	Principe d'apprentissage	18
2.2	Réseaux de neurones	20
2.2.1	Réseaux et algèbre	20
2.2.2	Modélisation universelle	23
2.2.3	Historique	25
2.2.4	Apprentissage profond et représentations distribuées	27
3	Entraîner un réseau de neurones par descente de gradient	30
3.1	Problème et modélisation	30
3.1.1	Données d'apprentissage	30
3.1.2	Choix du modèle	31
3.1.3	Estimer la qualité du modèle	31

3.1.4	Première simplification	32
3.2	Descente de gradient	32
3.2.1	Descente de gradient pour un paramètre	33
3.2.2	Descente de gradient pour plusieurs paramètres	40
3.2.3	Remarques	45
4	Descente de gradient stochastique et mise en pratique	45
4.1	Descente de Gradient et Réseau de Neurones	45
4.1.1	Données d'apprentissage	46
4.1.2	Un réseau de neurones simple	46
4.1.3	Descente de gradient et backpropagation	47
4.2	Définition et principe	50
4.3	SGD en pratique	53
4.3.1	Variance et Batch size	53
4.3.2	Momentum	53
4.3.3	Optimiseurs	54
	Bibliography	57

1 Introduction

1.1 Apprentissage automatique et usages

L'apprentissage machine, (*Machine learning*) est une discipline qui peut être définie à travers ces deux buts :

1. Extraire des motifs, identifier des structures dans les données, si possible en termes de propriétés statistiques.
2. Utiliser cette information pour faire des prédictions ou des inférences à partir de données jamais vues jusque là.

1.1.1 Définitions et motivations

Parmi les tâches fréquentes nous pouvons en identifier 2, la classification et la régression qui sont définies dans cette section.

1.1.1.1 Classification La classification consiste, pour des données $\{x_i\}_{i \in 1 \dots n}$ à prédire pour chaque élément x_i une catégorie discrète ou classe y_j parmi C classes $\{y_j\}_{j \in 1 \dots C}$. Cette classification peut être binaire si $C = 2$ ou multi-classe si $C > 2$. La détection de spam parmi des e-mails est un exemple de classification binaire ; un e-mail est soit un spam soit pas un spam. La classification d'animaux par espèce est un exemple de classification multi-classe pour une animal donné, son espèce est l'une des 10 millions d'espèce connue.

La classification peut également être mono-label ou multi-label. Dans l'exemple précédent, un animal n'appartient qu'à une espèce (mono-label) mais si il s'agit de classer une photo selon l'espèce de l'animal qui est dessus il est possible qu'une photo contienne plusieurs animaux, auquel cas on peut imaginer un système de classification qui indique la liste de toutes les espèces d'animaux présents sur la photo. La réponse du système est alors une liste de labels $(c_\alpha, c_\beta, c_\gamma)$ indiquant les classes des animaux.

1.1.1.2 Régression Parfois la réponse attendue n'est pas une catégorie discrète mais une (ou des) valeur numérique réelle. C'est le cas d'un système de prédiction météorologique qui doit indiquer une température (prédiction) pour une lieu précis (donnée d'entrée).

1.1.2 Modélisation et approches centrées sur les données

De manière générale, les systèmes d'apprentissage automatique utilisent des outils mathématiques pour traiter des données numériques afin de produire une sortie numérique. Cela nécessite donc la numérisation des données d'entrée grâce à un ou des capteurs (un microphone pour la parole, un appareil photo pour des systèmes de vision, etc.).

Une fois les données numérisées, elles se présentent sous forme de valeurs numériques $(x_1, x_2, x_3, \dots, x_n)$ auxquelles le système automatique applique une fonction $\mathcal{F}$ appelée aussi **modèle** telle que :

$$\mathcal{F}(x_1, x_2, x_3, \dots, x_n) = \hat{y} \quad (1)$$

où $\hat{y}$ est un entier borné dans le cas d'une classification mono-label, ou un réel dans le cas d'une régression. $\hat{y}$ est la prédiction réalisée par le système. L'objectif de l'apprentissage automatique consiste à trouver le modèle $\mathcal{F}$ qui fait en sorte que la prédiction $\hat{y}$ soit la plus proche possible de la vérité y quelles que soit les valeurs $(x_1, x_2, x_3, \dots, x_n)$.

Tout l'enjeu de l'apprentissage automatique consiste à déterminer la fonction, ou modèle, $\mathcal{F}$. En tant que fonction mathématique, $\mathcal{F}$ dépend souvent de paramètres. Si $\mathcal{F}$ est une droite (modèle linéaire) alors $\mathcal{F}$ est de la forme :

$$\mathcal{F}(x) = a \times x + b \quad (2)$$

où a et b sont les paramètres du modèle.

Selon les valeurs des paramètres choisis, le système va faire des prédictions plus ou moins correctes. La figure 1 montre des exemples de régression linéaire. Sur la figure de gauche, le modèle est très "éloigné" des données et ses prédictions seront certainement erronées alors que sur la figure de droite, le modèle est proche des données et devrait produire des prédictions correctes.

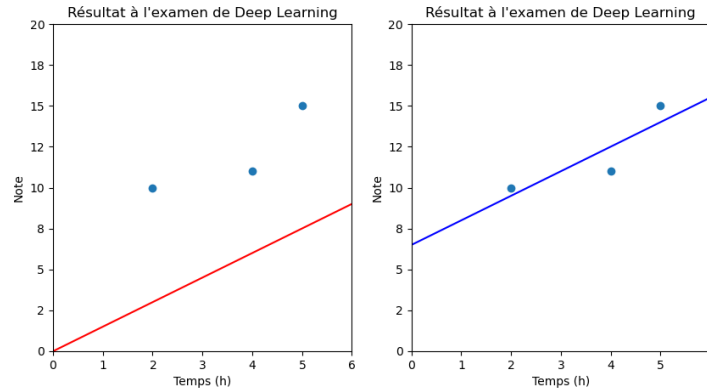


FIGURE 1 – Exemple de modèle pour une regression linéaire. Les points sont les données pour lequel le modèle doit faire une prédiction. Sur la figure de gauche, la droite du modèle est éloignée des données et fournira de mauvaises prédictions alors que sur la figure de droite le modèle est meilleur, sa droite est plus proche des points et les erreurs de prédiction devraient être moins importantes.

Question :

Pourquoi n'est il pas certain que le modèle de droite produira de meilleures prédictions que celui de gauche ?

Voir la section 1.2 sur les risques réel et empirique.

1.1.2.1 Comment apprendre les paramètres d'un modèle ?

1. en utilisant des règles, des lois physiques ;
2. en estimant les meilleures valeurs possibles à partir d'exemples (de données)

À l'heure actuelle, les approches les plus compétitives pour de nombreuses tâches estiment les paramètres à partir de données, ce sont des approches centrées sur les données, *data centric*. La suite de ce cours traite d'approches *data centric*.

1.2 Notion de risque

Nous avons vu juste précédemment ce qu'était une approche d'apprentissage automatique *data centric*. Dans cette section nous allons introduire les notions de risque réel et de risque empirique. Celles ci sont essentielles pour comprendre les problématiques d'apprentissage automatique *data centric*.

1.2.1 Définition de la tâche et de la fonction de coût

Dans cette section nous allons considérer une tâche de classification binaire (choix entre deux classes) pour laquelle notre modèle va devoir, étant données une entrée, ou observation, bi-dimensionnelle, prédire à quelle classe appartient cette observation. Les données d'entrée sont des vecteurs bi-dimensionnelles dont chaque coordonnée est un réel compris entre 0 et 1. Chaque point de l'espace d'entrée appartient à une classe "jaune" ou "vert". Il est possible de "mesurer" la classe à laquelle appartient un point mais le coût de cette mesure et le fait que l'espace est continu font que nous ne pouvons pas mesurer la classe de plus de quelques points.

La figure 2 illustre cette tâche. Étant donnée l'espace des données d'entrées, un carré de côté 1, il nous faut prédire à quelle classe : "jaune" (classe 0) ou "vert" (classe 1) appartient chaque point de coordonnées (x, y) . En réalité, mais nous ne pouvons pas le savoir, la structure de cet espace est celle reproduite dans la figure de droite. Le contour du carré est "vert" alors que le centre est "jaune", ainsi pour un point de coordonnée (x, y) nous pouvons mesurer sa classe. Dans l'exemple de cette figure, le point de coordonnées (x, y) est "jaune".

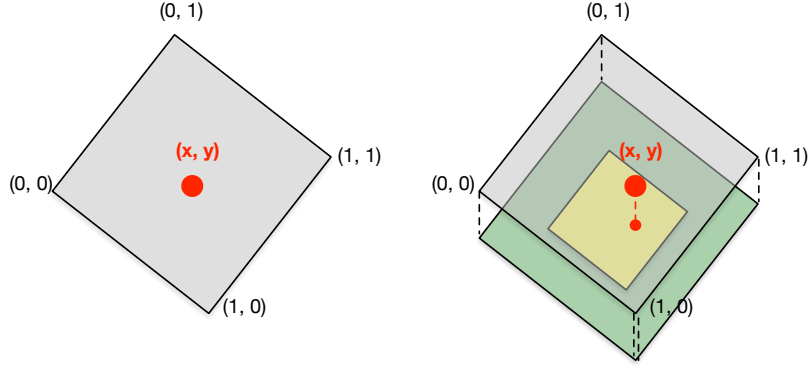


FIGURE 2 – Définition d'une tâche de classification binaire, à gauche l'espace d'entrée et à droite ce même espace avec, en dessous, sa structure réelle qui nous est inconnue et que nous souhaitons prédire.

1.2.2 Définition et différences entre risque réel et empirique

On définit une fonction de coût l de $[0, 1] \times [0, 1] \times \mathbb{R} \rightarrow \mathbb{R}_+$. Cette fonction de coût sert à mesurer l'erreur commise par le système automatique.

1.2.2.1 Risque réel Le **risque réel** est l'espérance de la fonction de coût sur la **distribution de données** $\mathcal{D}$ comme décrit par l'équation 3.

$$L \underbrace{\mathcal{D}}_{\substack{\text{Data} \\ \text{distribution}}} \underbrace{\widehat{(h)}}_{\substack{\text{Predictor} \\ \text{Expectation}}} = \underbrace{\mathbb{E}_{(x,y) \sim \mathcal{D}}}_{\text{Expectation}} \underbrace{[l((x_1, x_2, y), h)]}_{\text{Loss}} \quad (3)$$

où (x_1, x_2) sont les données d'entrée, y la classe du point correspondant et h le classifieur (modèle) utilisé pour prédire la classe. En pratique la distribution réelle des données, $\mathcal{D}$, est inconnue (rappelez-vous que dans l'exemple de la figure 2, le nombre de points dans le carré est infini et la structure vert/jaune est inconnue). Il n'est donc pas possible de calculer ce risque.

1.2.2.2 Risque empirique Étant donné un jeu fini de données, $(x_1^i, x_2^i)\}_{i \in [1, m]}$ (rappelez-vous que dans l'exemple de la figure 2, le nombre de points dans le carré est infini et la structure vert/jaune est inconnue), le **risque empirique** calculé sur ce jeu de données est la moyenne de la fonction de coût **sur ce jeu de données** comme indiqué par l'équation 4 .

$$L \underbrace{S}_{\text{Dataset}} \underbrace{\widehat{(h)}}_{\substack{\text{Predictor} \\ \text{Average}}} = \underbrace{\frac{1}{m} \sum_{i=1}^m}_{\text{Average}} \underbrace{[l((x_1^i, x_2^i, y_i), h)]}_{\text{Loss}} \quad (4)$$

Note

Contrairement au risque réel, défini dans l'équation 3, le risque empirique défini par l'équation 4 est une **somme finie** calculée sur des données pour lesquelles la valeur réelle de la classe a été "mesurée". La mesure de ces données a un coût.

Le but de l'apprentissage automatique est de déterminer le modèle (ou prédicteur), h qui minimise le risque empirique calculé sur le jeu de données S .

$$h_S = \operatorname{argmin}_{h \in \mathcal{H}} L_S(h) \quad (5)$$

où $\mathcal{H}$ est l'ensemble des modèles possibles et le risque empirique L_S est défini par :

$$L_S(h) = \frac{1}{m} \sum_{i=1}^m [l((x_i, y_i), h)] \quad (6)$$

1.2.2.3 Problématique Une fois le modèle trouvé, nous pourrions utiliser celui-ci pour prédire la classe, "jaune" ou "vert" pour n'importe quel point de l'espace d'entrée pour lequel nous n'avons pas effectué de mesure. Dans notre exemple, ce nombre de points est infini.

Dans un cas idéal, le risque réel doit être similaire au risque empirique (cf. équation 7), c'est-à-dire que notre modèle ne fait pas plus d'erreurs dans la vie réelle qu'il n'en a fait dans notre laboratoire pendant la phase de développement.

$$L_{\mathcal{D}(h)} \approx L_{S(h_S)} \quad (7)$$

Note

Mettez-vous à la place de votre client. Si vous calculez un risque empirique en laboratoire, par exemple vous mesurez que votre système fait 2% d'erreurs et le présentez ainsi à votre client. Lorsque ce dernier met votre système en production, il ne doit pas obtenir 20% d'erreurs !

1.2.3 Notion de sur-apprentissage

Revenons à notre exemple de classification binaire. La distribution réelle, et inconnue, des données est illustrée par la figure 3a. En pratique, on peut mesurer la classe de 6 points, comme illustré sur la figure 3b.

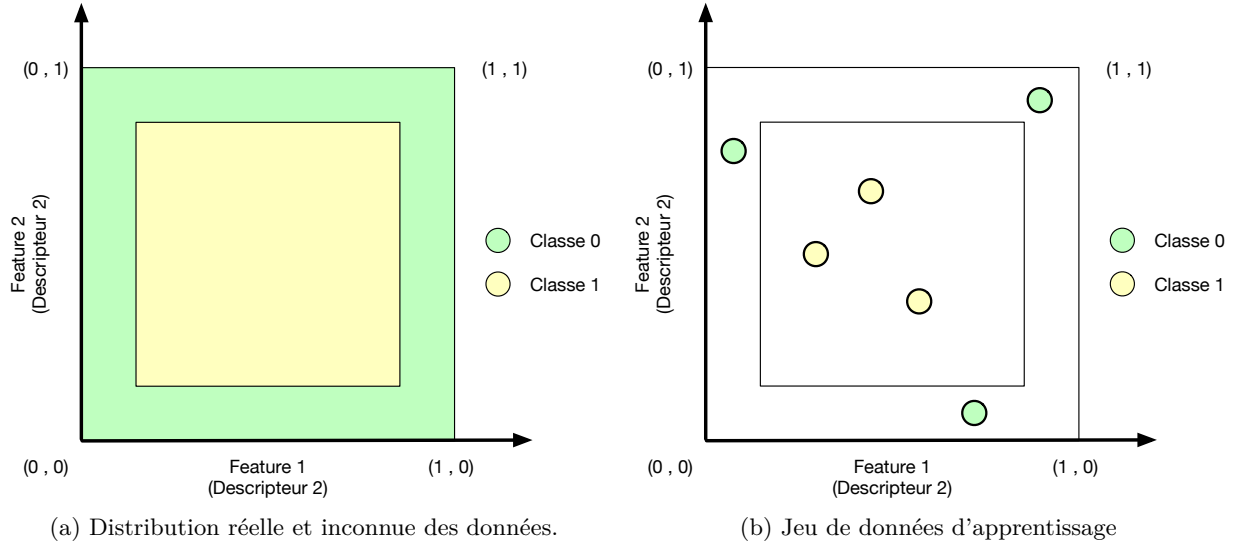


FIGURE 3 – Différence entre la distribution réelle et inconnue des données et le jeu de données fini d'apprentissage disponible pour estimer le meilleur modèle possible.

Étant données le jeu d'apprentissage de la figure 3b, le meilleur classifieur que nous pouvons obtenir est une table de correspondance définie par :

$$h(x) = \begin{cases} 1, & \text{si } h(x) = 1 \\ 0 & \text{sinon} \end{cases} \quad (8)$$

Avec ce classifieur, nous pouvons calculer le risque empirique sur notre jeu de données. Ce risque est :

$$\mathcal{L}_{\text{empirical}} = 0 \quad (9)$$

Le risque empirique est nul, mon système est parfait, je peux le vendre à mes clients. Mais que se passe-t'il lorsque le système est mis en production ?

N'importe que données d'entrée, qui n'appartient pas à mon jeu d'apprentissage est classée par mon modèle comme appartenant à la classe 0 ou "vert". En effet, le calcul du risque réel (si nous trichons et supposons que nous connaissons la distribution réelle des données) est :

$$\mathcal{L}_{\text{real}} = 0.5 \quad (10)$$

La moitié du temps (c'est la proportion de points jaunes dans mon espace) le système se trompe. En effet, les parties jaunes et vertes de notre espace ont la même surface et sont toutes les deux composées d'une infinité de points. Notre classifieur permet de trouver la bonne réponse pour 3 points parmi une infinité, ces bonnes réponses sont négligeables et l'erreur commise par notre classifieur est égale à la surface de la partie verte où le classifieur ne donne que 3 bonnes réponses

contre une infinité de mauvaises. Pour la partie jaune, le classifieur a toujours raison.

A retenir

Le mauvais choix du classifieur nous a amené à être (beaucoup) trop confiant et nous avons :

$$\begin{aligned}\mathcal{L}_{\text{real}} &= 0.5 \\ \mathcal{L}_{\text{empirical}} &= 0.0\end{aligned}$$

C'est qu'on appelle du **sur-apprentissage**. Ici nous observons un cas extrême pour lequel le modèle a **appris par cœur** les données d'apprentissage. En pratique cela peut souvent arriver si notre modèle a une *trop grande* capacité de mémoire. Pensez aux modèles actuels qui ont des milliards de paramètres.

1.2.4 Peut-on éviter le sur-apprentissage ?

L'une des raisons qui fait que notre modèle a sur-appris vient de la quantité de données d'apprentissage trop limitée. Supposons qu'on peut augmenter la taille de ce jeu d'apprentissage. Choisissons maintenant un classifieur moins *puissant* que la table de correspondance : un algorithme des K plus proches voisins.

Cet algorithme estime la classe d'un échantillon x comme suit :

$$h(x) = y_i \tag{11}$$

avec y_i la classe du plus proche voisin de x au sens de la distance d :

$$i = \operatorname{argmin}_i(d(x, x_i)) \tag{12}$$

Cet algorithme réduit le sur-apprentissage puisque :

$$\begin{aligned}\mathcal{L}_{\text{real}} &= 0.35 \\ \mathcal{L}_{\text{empirical}} &= 0.0\end{aligned}$$

En ajoutant un échantillon à notre jeu de données d'apprentissage (cf figure 5) nous obtenons :

Cet algorithme réduit le sur-apprentissage puisque :

$$\begin{aligned}\mathcal{L}_{\text{real}} &= 0.25 \\ \mathcal{L}_{\text{empirical}} &= 0.0\end{aligned}$$

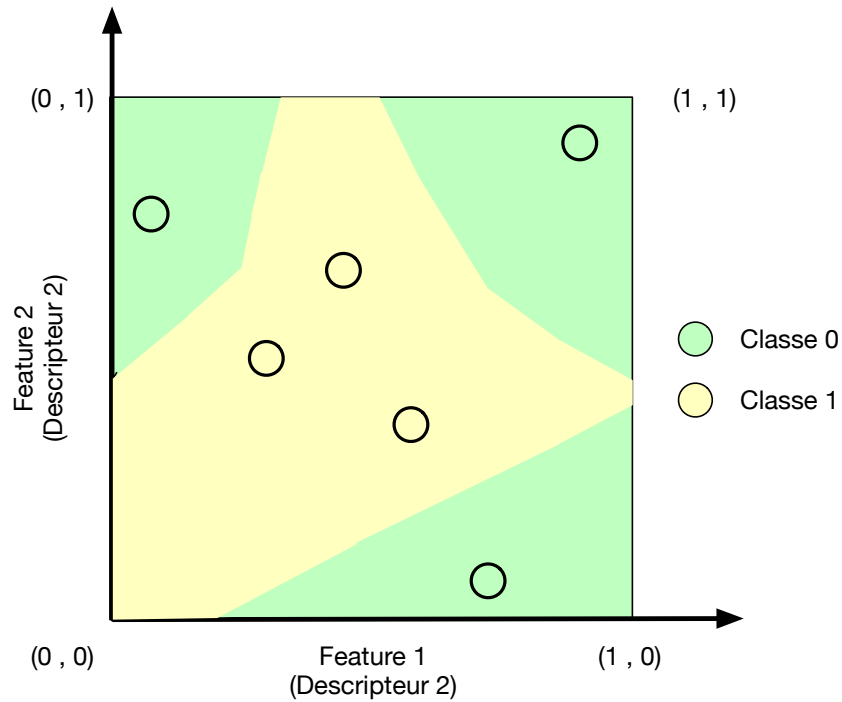


FIGURE 4 – En utilisant le jeu de données S comprenant 6 données, notre algorithme des K plus proches voisins avec $K = 1$ estime la distribution des données $\mathcal{D}$ ainsi.

En utilisant beaucoup plus de données, comme sur la figure 6

Est-ce réaliste ?

En supposant que l'on souhaite mesurer des données à interval régulier, le nombre de données pour paver l'espace augmente de façon exponentielle avec la dimension de l'espace, comme illustré par la figure 7. L'acquisition de données d'apprentissage pour paver l'espace des données possible n'est pas réaliste, il faut trouver un moyen de **généraliser**.

- pour $d = 1$: 10 échantillons
- pour $d = 2$: 100 échantillons
- pour $d = 3$: 1000 échantillons

1.2.5 Notion de manifold

En pratique, il n'est pas possible de recueillir assez de données pour garantir un pavage régulier de l'espace des données. Mais à bien y regarder, en avons nous besoin ?

Pour une tâche simple comme la reconnaissance de chiffres manuscrits, supposons que les images son en noir et blanc et de dimension 28×28 comme illustré par la figure 8.

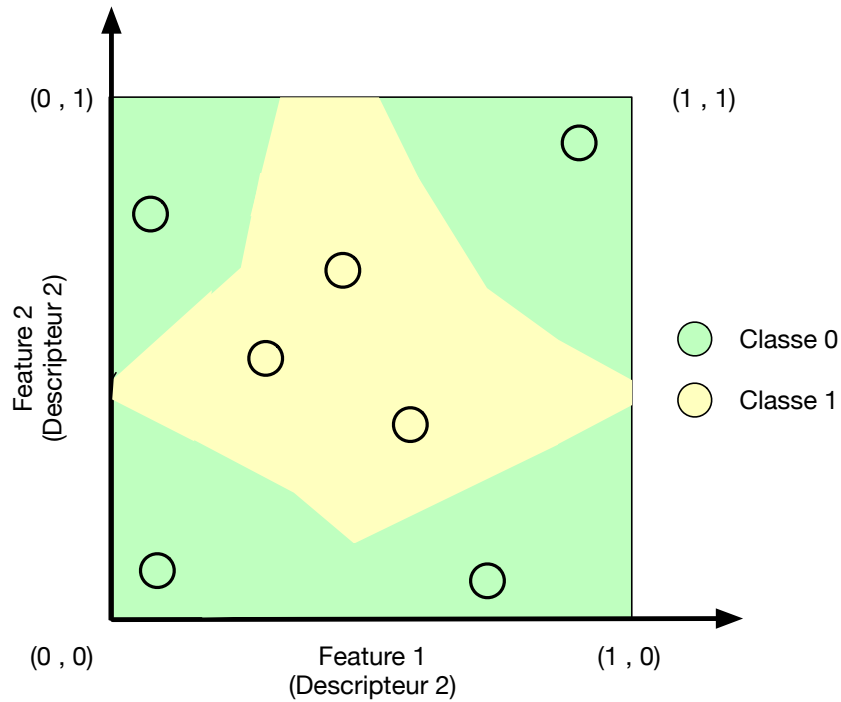


FIGURE 5 – En augmentant notre jeu de données S pour atteindre 7 données, notre algorithme des K plus proches voisins avec $K = 1$ estime la distribution des données $\mathcal{D}$ ainsi.

La probabilité de trouver une image de chiffre en pavant régulièrement l'espace des images de dimension 28×28 est proche de 0. En fait, l'ensemble des chiffres respecte une structure bien particulière. C'est souvent le cas en apprentissage automatique : les données appartiennent à un sous-espace particulier dont la structure est appelée *manifold*, cf. figure 9.

1.2.6 Conclusion

En apprentissage automatique, on a souvent tendance à dire que **Better data is more data**. Mais en pratique :

- la collecte de données coûte cher
- les données coûtent cher à stocker (disques durs, électricité, sauvegarde)
- les données coûtent cher à traiter (CPUs, GPUs, RAM)
- parfois aucune donnée additionnelle n'est disponible (en médecine on ne peut pas opérer des patients juste pour collecter des données, dans l'industrie, il n'est pas forcément possible d'arrêter des chaînes de production)
- les données sont parfois sensibles et la loi nous oblige à en limiter la collecte (RGPD, éthique)

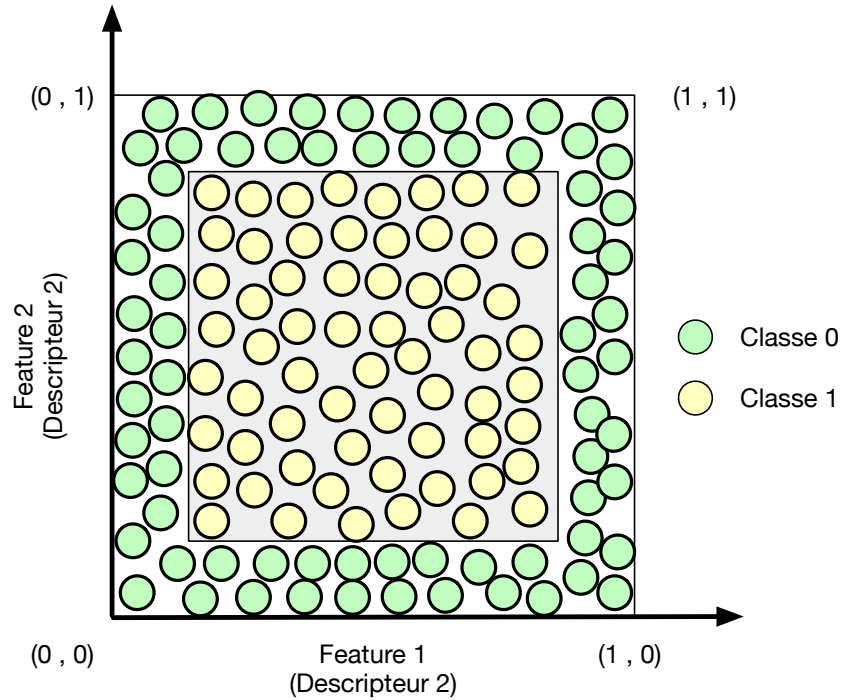


FIGURE 6 – En augmentant notre jeu de données S , notre algorithme des K plus proches voisins avec $K = 1$ peut réduire le risque empirique jusqu'à tendre vers le risque réel.

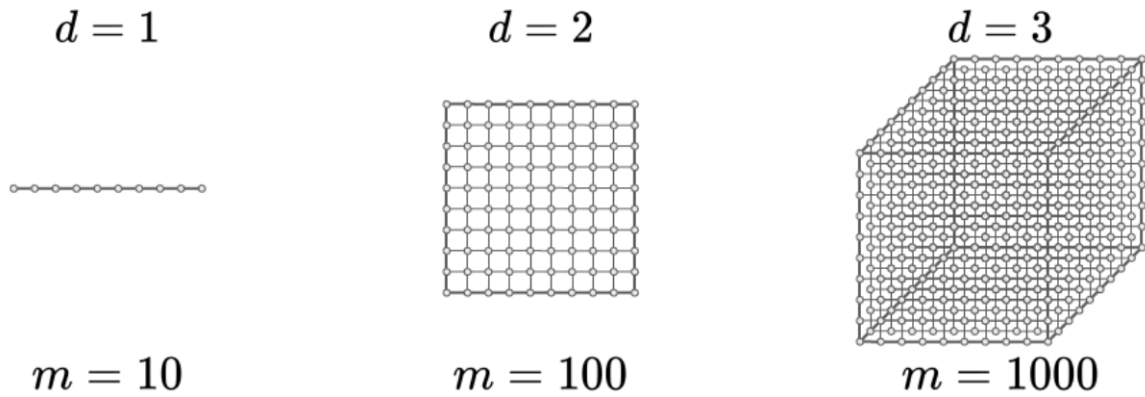


FIGURE 7 – Le nombre de données pour paver l'espace augmente de façon exponentielle avec la dimension de l'espace.

Il faut donc trouver des moyens d'éviter le sur-apprentissage sans augmenter la quantité de données d'apprentissage. Nous verrons ces moyens plus tard dans le cours.

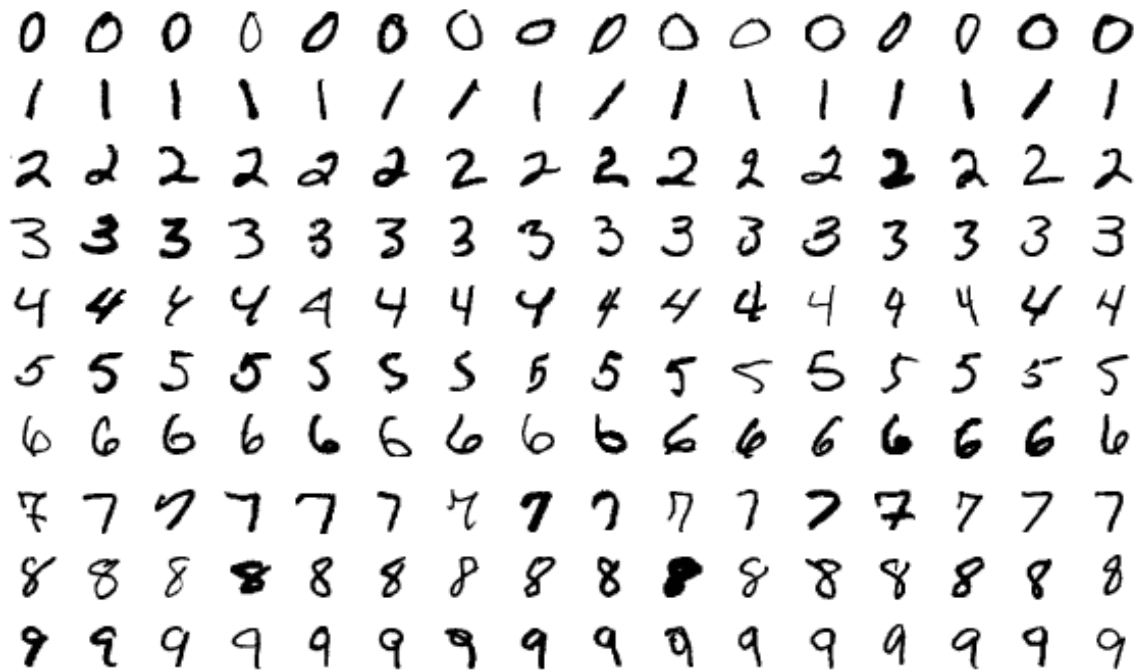
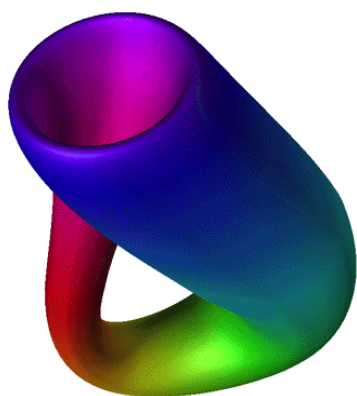
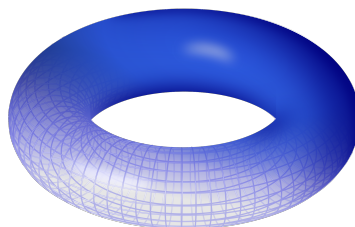


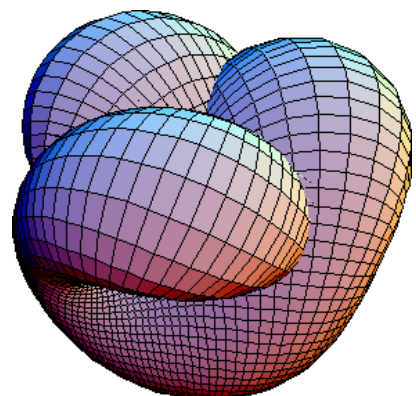
FIGURE 8 – Exemples de chiffres dans des images de dimension 28×28 .



(a) 1



(b) 2



(c) 3

FIGURE 9 – Exemples de manifolds complexes dans un espace de dimension 3.

2 Neurones artificielles

Les neurones artificielles sont un type de modèle parmi beaucoup d'autres. Leur histoire remonte au milieu du XX^e siècle et a connu de nombreux rebondissements. Depuis 2006, des classifieurs, associés à la puissance de calcul des cartes graphiques (GPUs) ont permis à l'apprentissage automatique d'atteindre des performances remarquables, suffisantes pour de nombreuses applications pratiques.

2.1 À l'origine était le perceptron

Introduit dans les années 1940 et 1950, le perceptron est à l'origine une machine imaginée pour de la reconnaissance visuelle. Le perceptron repose sur l'approche connectionniste qui considère que l'information est un signal.

2.1.1 Modèle du perceptron

Le perceptron s'inspire du neurone biologique même si il en est une version très simplifiée.

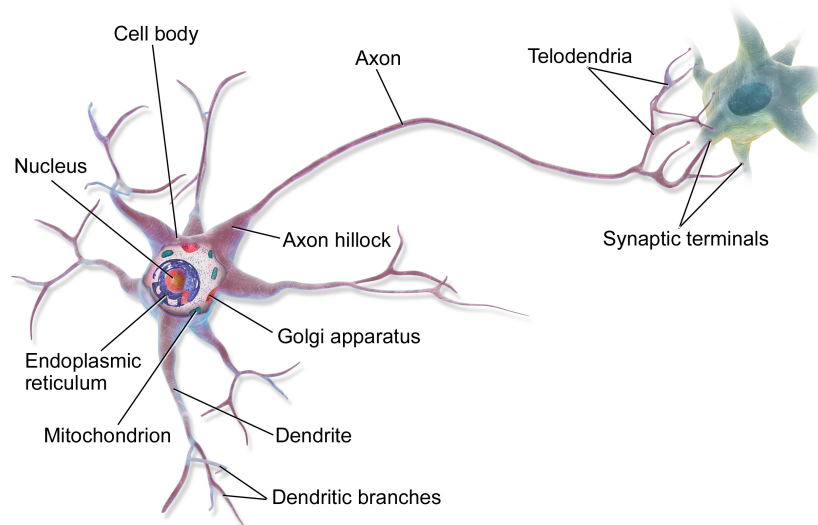


FIGURE 10 – Représentation d'un neurone biologique D'après https://en.wikipedia.org/wiki/Neural_circuit, vu le 01/08/2023.

Le perceptron reprend le principe selon lequel un neurone reçoit des stimulations à travers plusieurs dendrites. Ces stimulations sont traitées par la cellule (de façon très complexe car le traitement s'effectue au niveau de la dendrite, du cœur de la cellule et de l'axone), avant que l'axone ne renvoie la réponse générée par ce neurone. Si en biologie le processus est très complexe et encore mal compris, en apprentissage automatique le perceptron prend une forme assez simple, décrit par la figure

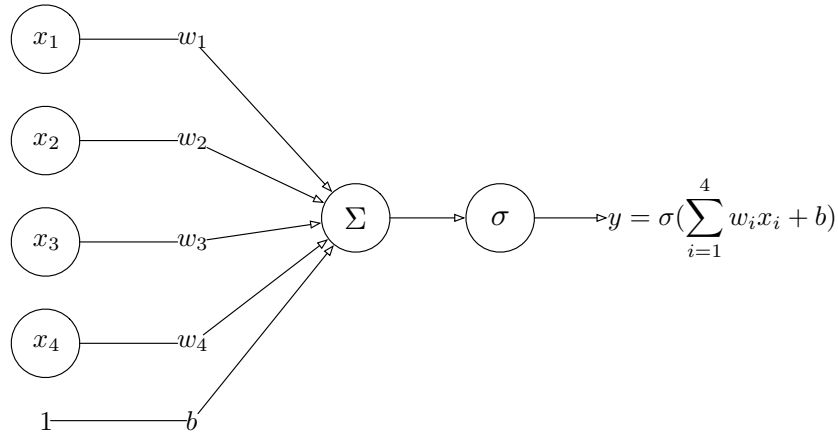


FIGURE 11 – Représentation d’un neurone artificielle, une entrée multidimensionnelle $(x_1, x_2, x_3, x_4, 1)$ est multipliée par des poids (w_1, w_2, w_3, w_4, b) avant d’être sommée et de passer dans une fonction d’activation σ .

Définitions

Dans la figure 11 :

- (x_1, x_2, x_3, x_4) est une **entrée** multi-dimensionnelle
- (w_1, w_2, w_3, w_4) sont les **poids** du modèle
- b est le **biais**
- σ est la **fonction d’activation**
- y est la **sortie**

La fonction d’activation d’un perceptron est fonction échelon unité illustrée sur la figure 13a qui prend la valeur 1 lorsque la somme de entrées pondérées par leur poids respectif dépasse 0 et prend la valeur 0 sinon.

2.1.2 Fonction d’activation et limitations

La partie linéaire du perceptron permet d’approximer une fonction linéaire des données d’entrée. Dans le cas d’un perceptron à 2 entrées, la fonction linéaire correspond à un hyperplan (une surface plane) qui divise l’espace en deux parties. Un exemple est illustré par la figure 12.

Le code python correspondant est donné ci-dessous et la surface est tracé avec la méthode `plot_surface` de Matplotlib.

```
def func(x_0, x_1):  
    w_0 = -1.5  
    w_1 = 0.2  
    b = 0.2  
    y = x_0 * w_0 + x_1 * w_1 + b  
    return y
```

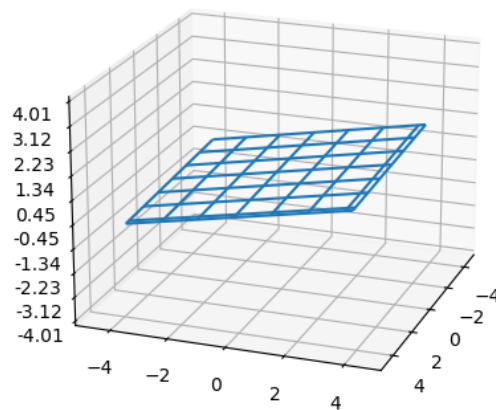
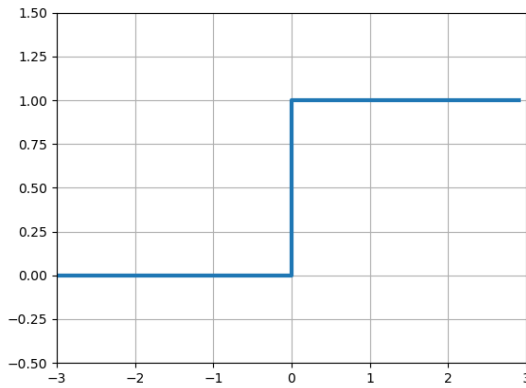


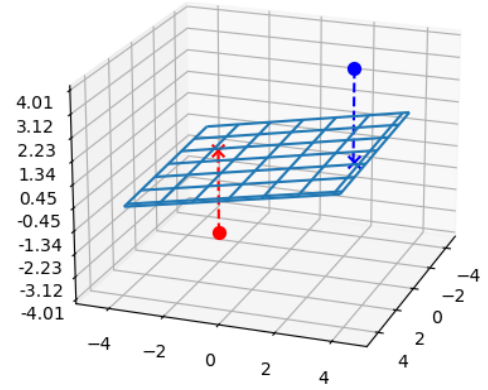
FIGURE 12 – Hyperplan obtenue par un perceptron donc les poids sont $w = [-1.5, 0.2]$ et le biais $b = 0.2$.

Un perceptron permet de découper l'espace des entrées en deux parties séparées par un hyperplan comme celui représenté sur la figure 13b. L'ajout d'une fonction d'activation de type *échelon* permet, pour une entrée $A = (\alpha_0, \alpha_1)$ d'obtenir en sortie du perceptron une valeur binaire $\{0; 1\}$ selon le signe de $func(\alpha_0, \alpha_1)$ (voir Figure 13b).

La principale limite du perceptron vient de son incapacité à modéliser des fonctions non-linéaires. Cette limitation peut être levée en remplaçant la fonction d'activation *échelon* par une fonction non-linéaire.



(a) Fonction échelon unité.



(b) Hyperplan de séparation dans un espace à 3 dimensions

FIGURE 13 – À gauche, fonction d’activation d’un perceptron et à droite illustration d’un hyperplan dans un espace à 3 dimensions.

2.1.3 Non-linéarités

Le perceptron peut également être utilisé pour des tâches de régression ou pour prédire une pseudo-probabilité en remplaçant la fonction d’activation *échelon* par une fonction non-linéaire telle qu’une sigmoïde (cf. figure 14a).

La figure 14b représente la sortie d’un neurone pour une entrée bi-dimensionnelle. Pour chaque point du plan, le neurone renvoie une valeur entre 0 et 1 qui peut être utilisée pour approximer une probabilité.

2.1.4 Principe d’apprentissage

Un neurone artificiel permet de modéliser une fonction grâce à une fonction linéaire paramétrique et d’une fonction d’activation. La difficulté consiste à déterminer les paramètres du modèles qui permettent d’obtenir la meilleure modélisation possible d’un phénomène. Cette section illustre de façon très simplifiée le mécanisme d’apprentissage des paramètres d’un neurone artificiel.

La tâche que nous souhaitons réaliser est une classification d’images. Des images binaires (noir et blanc) de dimension 8×8 constituent nos données d’entrée et ces images représentent

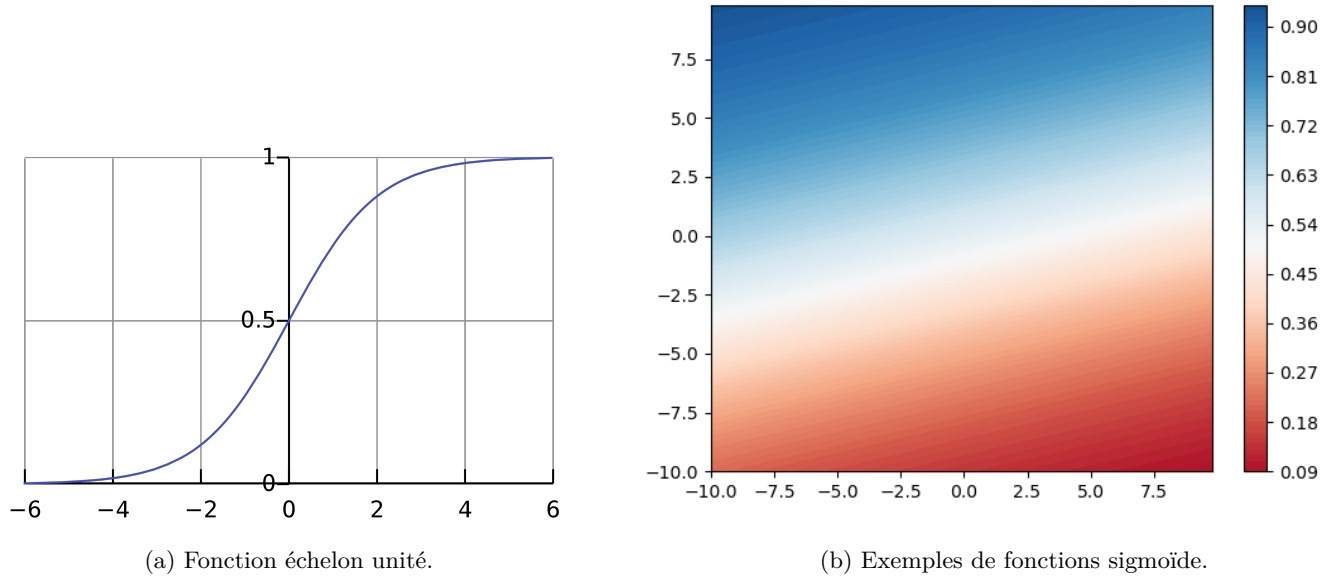


FIGURE 14 – À gauche, exemples de fonctions sigmoïde et à droite sortie d'un neurone à entrée bi-dimensionnelle doté de cette fonction d'activation.

soit la lettre A soit la lettre B . Notre neurone doit déterminer laquelle de ces deux lettres et renvoyer 1 si la lettre est un A et -1 si la lettre est un B . Pour cet exemple simple nous considérons que le neurone ne dispose pas de biais ni de fonction d'activation.

La figure 15 représente un neurone simplifié dont les entrées sont les valeurs des pixels de l'image. Les poids de ce neurone sont initialisés à 0 de façon arbitraire. La sortie de ce neurone est donc 0 pour l'image qui est présentée à ce neurone.

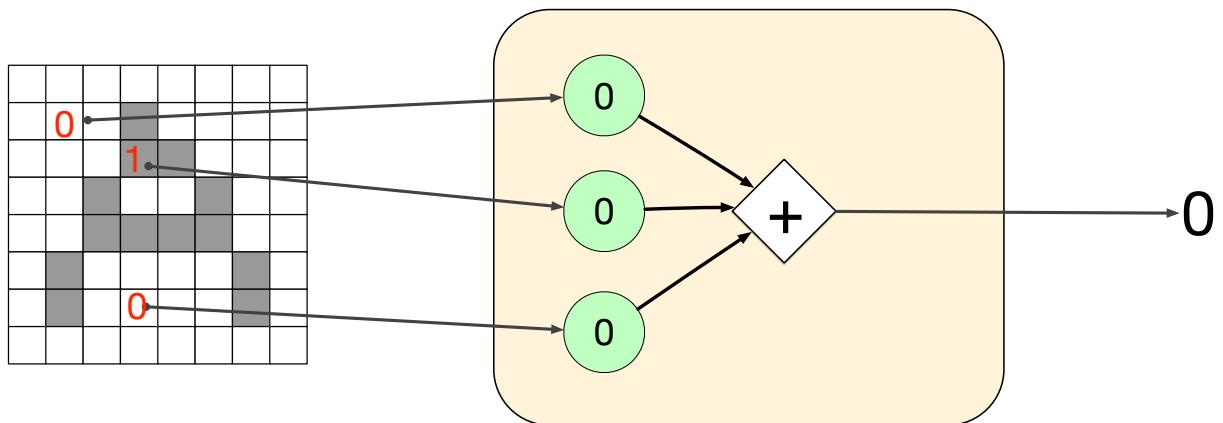


FIGURE 15 – Initialisation des poids d'un neurone simple.

Comme la sortie du neurone n'est pas la sortie attendue, les paramètres de celui-ci (les poids

seuls ici) sont modifiés afin de faire tendre la sortie vers la réponse attendue : 1.

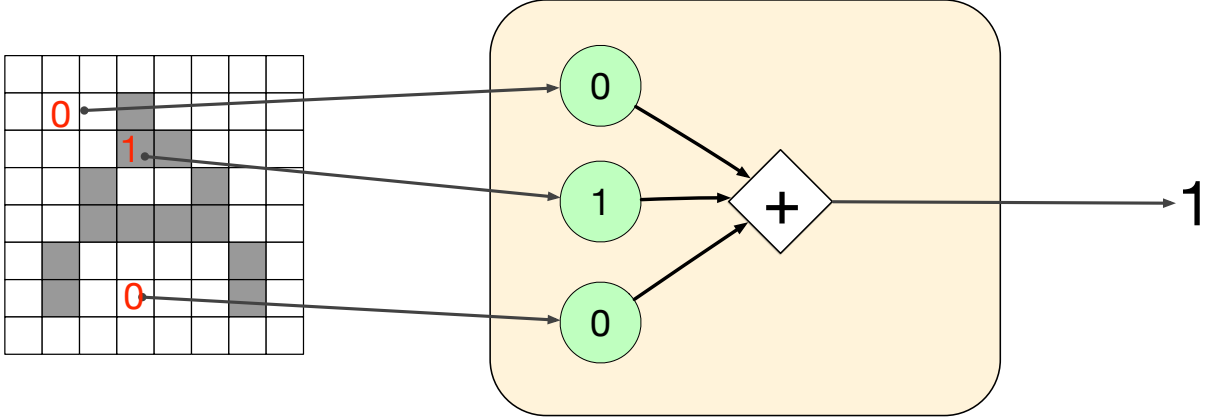


FIGURE 16 – Initialisation des poids d'un neurone simple.

Une nouvelle version du neurone est illustrée sur la figure 16. Comme ce neurone doit également retourner la valeur -1 lorsque la lettre est un B , nous lui présentons une de ces images et calculons la sortie. La sortie n'étant pas celle attendue, les poids sont modifiés pour la faire tendre vers la réponse correcte. Le processus itératif est réitéré jusqu'à satisfaire un critère déterminé au préalable. La figure 17 illustre ce processus.

2.2 Réseaux de neurones

Nous avons vu dans la section 2.1 qu'un neurone artificielle est une fonction mathématique qui traite une entre multi-dimensionnelle $x = (x_0, \dots, x_n)$ pour calculer une valeur de sortie y .

2.2.1 Réseaux et algèbre

Nous allons ici définir la notion de **couche de neurones**. Considérons qu'une même entrée peut être traitée en parallèle par plusieurs neurones artificielles comme sur la figure 18

Chaque neurone, i , produit une sortie $y_i = \sigma(\sum_{j=0}^n w_{i,j} \times x_j + b_i)$. Nous obtenons alors :

$$y_0 = \sigma(w_{0,0} \cdot x_0 + w_{0,1} \cdot x_1 + w_{0,2} \cdot x_2 + w_{0,3} \cdot x_3 + b_0)$$

$$y_1 = \sigma(w_{1,0} \cdot x_0 + w_{1,1} \cdot x_1 + w_{1,2} \cdot x_2 + w_{1,3} \cdot x_3 + b_1)$$

$$y_2 = \sigma(w_{2,0} \cdot x_0 + w_{2,1} \cdot x_1 + w_{2,2} \cdot x_2 + w_{2,3} \cdot x_3 + b_2)$$

Ce système d'équation peut être écrit sous forme matricielle de la façon suivante :

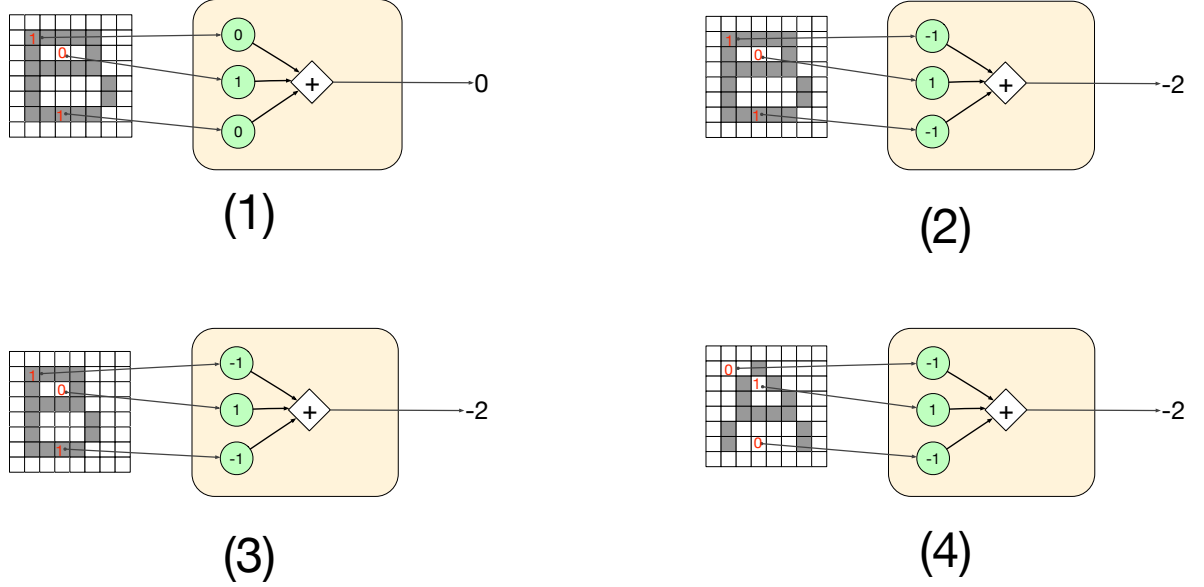


FIGURE 17 – Itérations successives d'apprentissage des paramètres d'un neurone artificiel. La sous-figure (1) montre qu'après une itération le neurone ne renvoie pas la bonne réponse pour un B . La valeur de ses poids est mise à jour pour obtenir un résultat négatif (2) et on voit sur la sous-figure (3) que ce neurone renvoie la même valeur pour une instance de B différente. En revanche, la sous-figure (4) montre qu'après modification de ses poids, le neurone renvoie une réponse erronée pour un nouveau A . Cette exemple illustre la difficulté de l'apprentissage des poids.

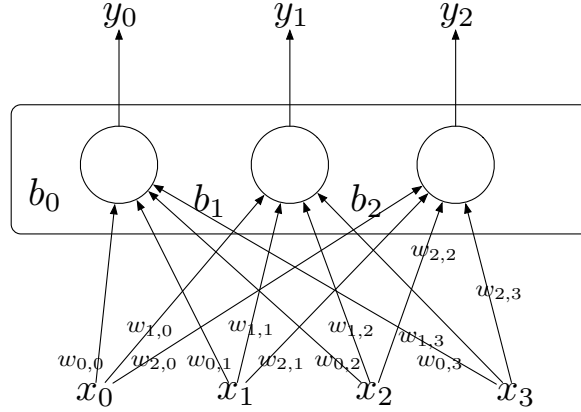


FIGURE 18 – Couche de neurones traitant une même entre multidimensionnelle $x = (x_0, \dots, x_3)$ pour générer plusieurs sorties (y_0, y_2) .

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \end{bmatrix} = \sigma \left(\begin{bmatrix} w_{0,0} & w_{0,1} & w_{0,2} & w_{0,3} \\ w_{1,0} & w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,0} & w_{2,1} & w_{2,2} & w_{2,3} \end{bmatrix} \times \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix} \right)$$

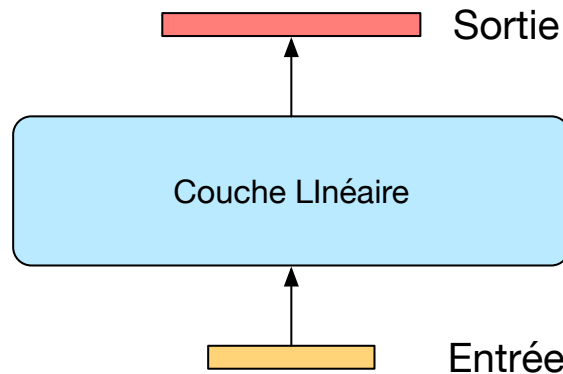


FIGURE 19 – Représentation d’une couche de neurones traitant une même entrée multidimensionnelle $x = (x_0, \dots, x_n)$ pour générer plusieurs sorties $\{y_0, y_p\}$.

L’entrée d’une couche est un vecteur et la sortie est un autre vecteurs (la dimension de ce dernier est égale au nombre de neurones dans la couche). Par la suite nous représenterons ce type de couche de neurones de la façon suivante comme sur la figure 19.

L’organisation des neurones artificiels en couche et la modélisation vectorielle qui en résulte permet d’implémenter simplement ces calculs et de bénéficier de l’accélération due à la vectorisation sur CPU ou sur GPU. Ainsi, en utilisant le module `PyTorch`¹, nous instancions une couche de neurones de la façon suivante :

```
import torch
layer = torch.nn.Linear(in_features=4, out_features=3, bias=True)
```

Comme son nom l’indique, cette couche implémente la partie linéaire des neurones d’une couche, sans ajouter de fonction d’activation. Celle-ci sera ajoutée dans un deuxième temps. Nous pouvons vérifier la dimension des matrices de poids :

```
layer.weight.shape
```

```
torch.Size([3, 4])
```

Leur valeurs :

```
layer.weight
```

Parameter containing:

1. <https://pytorch.org/>

```
tensor([[ -0.1055,  0.1928, -0.4196, -0.2930],
        [ 0.3316,  0.2521, -0.2464,  0.3881],
        [-0.2764, -0.4888,  0.0599,  0.3715]], requires_grad=True)
```

et faire de même pour le vecteurs de biais :

```
layer.bias.shape
```

```
torch.Size([3])
```

```
layer.bias
```

Parameter containing:

```
tensor([ 0.3533, -0.1728, -0.1696], requires_grad=True)
```

L'assemblage de plusieurs neurones, généralement organisés par couches successives, permet de modéliser des fonctions très complexes.

En 1986, David Rumelhart, Geoffrey Hinton et Ronald Williams publient “*Learning representations by back-propagating errors*”, qui introduit :

- les couches cachées
- la rétro-propagation (*Backpropagation*)

qui ont permis le développement de nombreuses architectures. Le perceptron multi-couche est organisé en couches successives où les entrées d'un neurone appartenant à la couche i sont les sorties des neurones de la couche $i - 1$. Un perceptron multi-couche contient au moins une couche cachée.

2.2.2 Modélisation universelle

Les perceptrons et perceptrons multi-couches souffrent d'une limitation importante quant aux types de fonctions qu'ils peuvent modéliser. Or, le théorème d'approximation universelle nous apprend qu'un réseau de neurones, si il est doté d'un nombre de neurones *suffisant* et d'une non-linéarité est en mesure de modéliser un grand nombre de fonctions très complexes.

Théorème

Le théorème d'approximation universelle indique qu'un réseau à propagation avant d'une seule couche cachée contenant un nombre fini de neurones, ce qu'on appelle un perceptron multi-couche, peut approximer des fonctions continues sur des sous-ensembles compacts

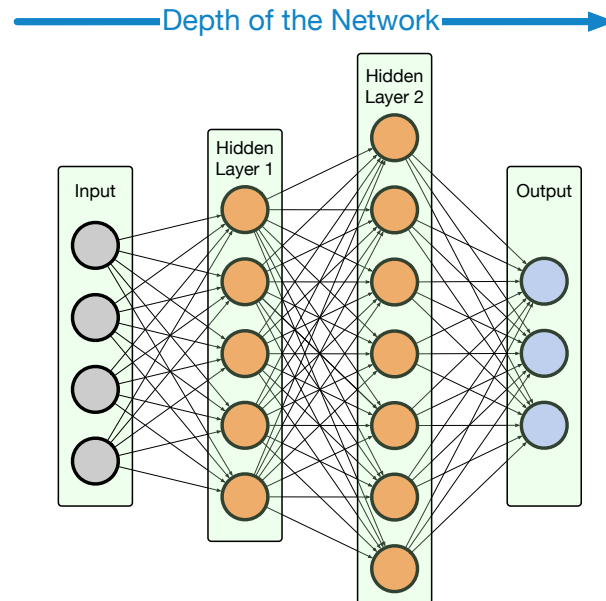
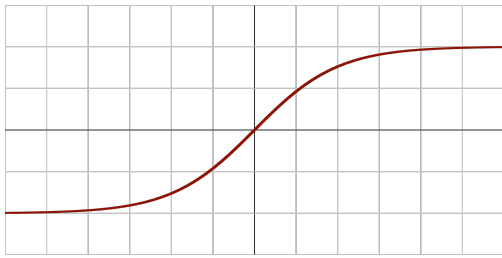


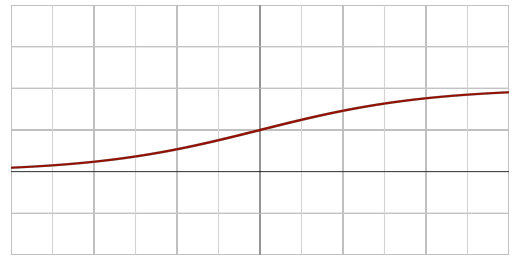
FIGURE 20 – Exemple d'architecture d'un perceptron multi-couche.

de $\mathbb{R}^n$.

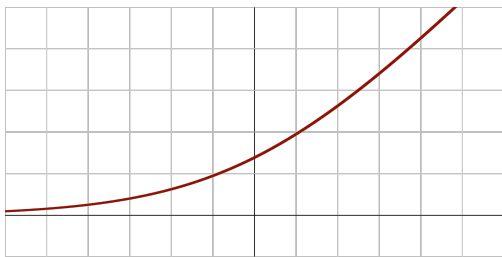
Les fonctions d'activation non-linéaire offrent donc un énorme potentiel à ces modèles pour modéliser toutes sortes de fonctions. La figure 21 illustre quelques une des fonctions d'activation les plus utilisées. Même si il est possible de modéliser un même phénomène avec plusieurs fonctions d'activations, le choix de cette fonction est essentiel pour l'apprentissage profond. Il est particulièrement important de garder en tête les zones de linéarité et de saturation de ces fonctions.



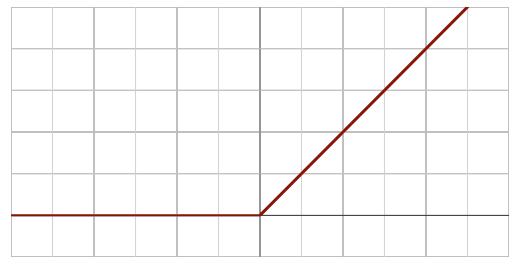
Hyperbolic tangent



Sigmoid



Softplus



ReLU

FIGURE 21 – Exemples de fonctions d'activation fréquemment utilisées.

2.2.3 Historique

Depuis ces travaux, de nombreuses architectures ont été proposées et seront traitées dans la suite de ce cours :

- Réseaux à propagation avant
- Réseaux convolutionnels
- Restricted Boltzman Machines (non-supervisé)
- Réseaux récurrents (modélisation de séquences RNN, LSTM, GRU)
- Mécanismes d'attention (Transformers)
- Graph Neural Networks

La figure 22 présente une frise chronologique des principales avancées dans le domaine de l'apprentissage profond.

Deep Learning timeline

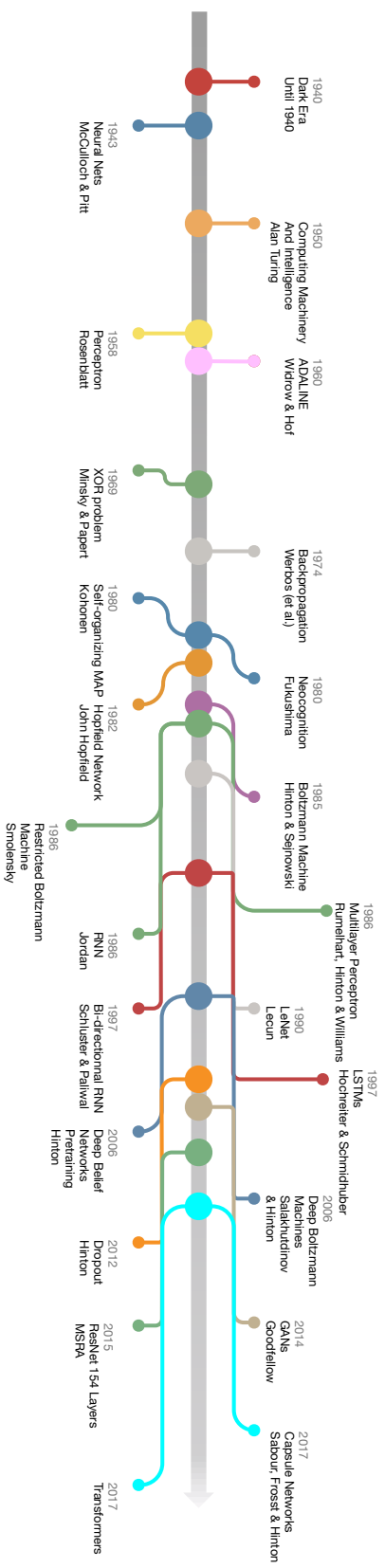


FIGURE 22 – Frise chronologique des principales avancées dans le domaine des réseaux de neurones artificiels.

2.2.4 Apprentissage profond et représentations distribuées

Avant d'aller plus loin dans les techniques d'apprentissage des réseaux de neurones, nous allons dans cette section illustrer les principales motivations de l'apprentissage profond. Les réseaux de neurones profonds présentent plusieurs avantages :

- Ils offrent des représentations dans des espaces de très grande dimension (plusieurs millions ou milliards de paramètres), qui permettent d'obtenir des **représentations de très grandes dimensions pour séparer les informations**.
- Il permettent d'obtenir des **représentations distribuées**
- Ils peuvent avoir des **architectures spécifiques** adaptées aux tâches visées (c'est l'objet de la suite du cours)

2.2.4.1 Extraction de descripteurs La figure 23 illustre une chaîne de traitement classique en apprentissage automatique. Les données d'entrée brutes sont traitées pour extraire des descripteurs qui sont ensuite utilisés en entrée de système automatiques pour de la classification comme dans l'exemple de la figure 23 ou pour une régression. Le modèle appris à partir de ces descripteurs est ensuite utilisé en inférence pour traiter de nouvelles données.

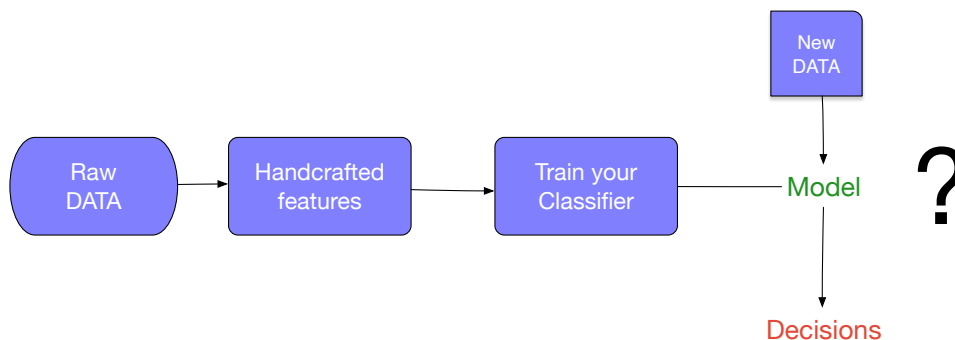


FIGURE 23 – Architecture classique d'un système d'apprentissage automatique.

Les modèles automatiques sont généralement peu efficaces pour traiter des données brutes car ces données sont généralement :

- de grande dimension
- redondantes
- bruitées

Obtenir de bonnes performances nécessite donc de représenter ces données par des descripteurs définis et optimisés par des humaines. Historiquement, tout un pan de la recherche en apprentissage automatique s'est intéressé au développement des meilleurs descripteurs possibles.

Les descripteurs, ou *features*, développés reposent la plupart du temps sur des concepts physique provenant de la compréhension des données par les humains. Dans le domaine de l'audio,

les descripteurs utilisent des représentations temps/fréquences reposant sur la transformée de Fourier (ou d'autres transformées) et sur la modélisation de l'appareil auditif humain (échelle MEL). Cependant, ces descripteurs, si ils sont interprétables par des humains, ne garantissent pas des résultats optimaux lorsqu'ils sont traités pas des systèmes automatiques.

Une hypothèse qui a motivé l'utilisation de l'apprentissage profond consiste à penser qu'un réseau de neurone profond peut lui même apprendre des descripteurs (ou représentations) des données qui soient plus adaptés car appris avec un critère automatique qui prend en compte la tâche finale. Les descripteurs ainsi obtenus peuvent remplacer l'approche empirique comme montré dans la figure 24.

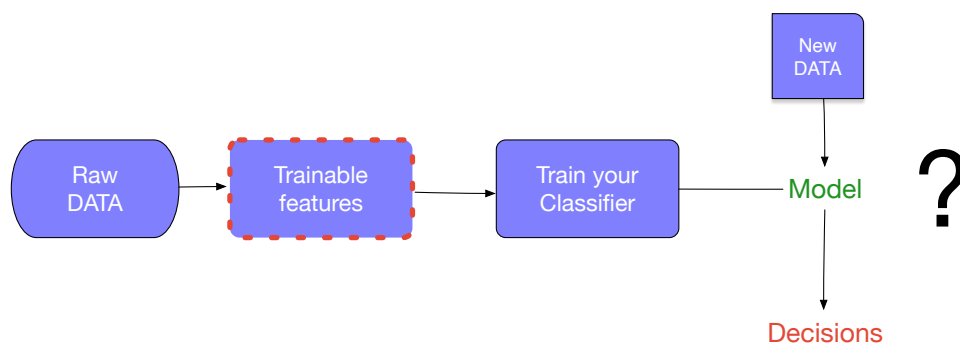


FIGURE 24 – Architecture classique d'un système d'apprentissage automatique.

2.2.4.2 Représentations distribuées Pour de nombreuses tâches, les descripteurs développés par des humains utilisent des concepts locaux et éparés. Considérant que nous disposons d'une mémoire limitée pour stocker une liste de descripteurs nous pouvons décrire les 4 formes géométriques de la figure 25 comme indiqué sur cette figure. Sur cette figure, chaque forme est

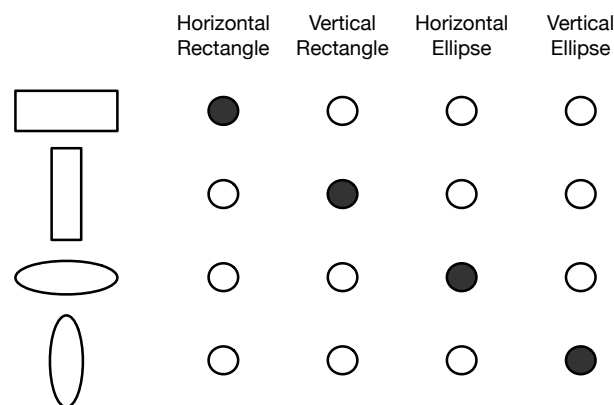


FIGURE 25 – Illustration de la description de 4 formes géométriques grâce à des représentations non-distribuées.

décrite précisément par un descripteur. Ce type de description présente plusieurs défauts :

- le nombre de descripteur augmente avec le nombre de formes géométriques à décrire ;
- les descriptions de chaque forme ne donne aucune information sur d'éventuels liens entre les formes

Dans l'idéal nous souhaiterions des descripteurs qui permettent de créer du lien entre les formes à décrire. Nous souhaitons :

- que la mémoire disponible stockent des informations conceptuelles ;
- que les descripteurs ne soient pas spécifiques à une seule forme ;
- que l'information d'une seule forme soit distribuée dans le modèle.

Une telle représentation des mêmes 4 formes est donnée dans la figure 26

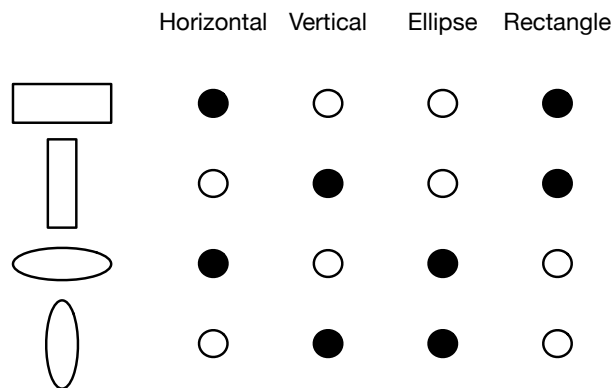


FIGURE 26 – Illustration de la description de 4 formes géométriques grâce à des représentations distribuées.

Un autre avantage de ces descripteurs qui utilisent des concepts vient du fait qu'il est maintenant possible de décrire des formes qui n'avaient encore pas été observées. La figure 27 montre ce qui pourrait être obtenu pour décrire un cercle à partir des descripteurs appris sur les 4 formes précédentes.

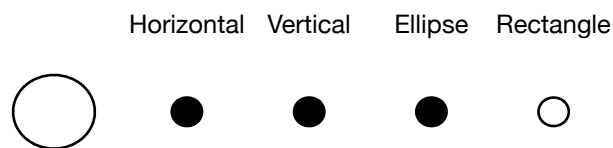


FIGURE 27 – Illustration de la description d'une forme jamais observée grâce à des représentations distribuées. un cercle correspond aux concept d'ellipse simultanément horizontale et verticale.

Ce phénomène correspond en partie à ce qui se passe dans un réseau de neurones. Plusieurs neurones du modèle peuvent être activés par un même concept. De plus, les réseaux de neurones permettent d'apprendre des descripteurs à différents niveaux en fonction de la profondeur de la couche à laquelle ces descripteurs sont extraits.

3 Entraîner un réseau de neurones par descente de gradient

3.1 Problème et modélisation

Un problème courant en apprentissage automatique consiste à prédire une valeur à partir d'observations : la régression. Pour faire une régression, il s'agit d'utiliser un jeu de données s'apprenant afin d'estimer un modèle théorique que nous pourrions utiliser par la suite pour prédire des valeurs que nous n'avons pas encore observées.

3.1.1 Données d'apprentissage

Notre tâche consiste à prédire la note (sur 20) que nous obtiendrons à l'examen de *Deep Learning* en fonction du temps (en heures) que nous passerons à travailler cette matière. L'année dernière, nous avons collecté des données à partir de trois étudiants et obtenu les données représentées sur la figure 28.

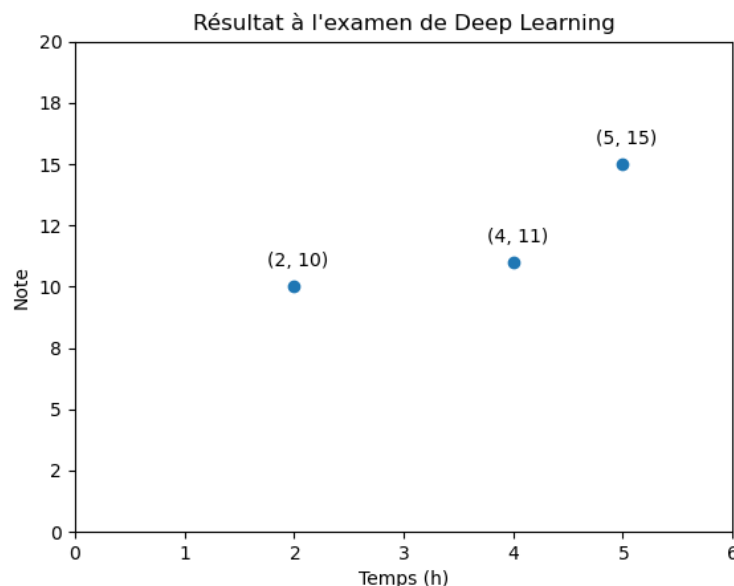


FIGURE 28 – Données d'apprentissage collectées auprès de 3 étudiants.

Nous voyons sur cette figure qu'un étudiant qui a travaillé 2 heures a obtenu la note de 10, le second qui a travaillé 4 heures a obtenu 11 et le dernier a obtenu 15 après 5 heures de travail.

3.1.2 Choix du modèle

Nous supposons que la note est une fonction linéaire du temps de travail. Selon cette hypothèse (qui n'est que pure spéculation), la note obtenue vérifierait l'équation suivante :

$$Note = Pente \times t + Connaissance \quad (13)$$

Selon ce modèle, la note obtenue ne dépend que de deux **paramètres** de la fonction : la *Connaissance*, qui correspond à la note obtenue sans travail et la *Pente*. Une fois ces deux paramètres connus, nous pouvons calculer, c-à-d **prédire**, la note obtenue pour n'importe quel durée de révision.

3.1.3 Estimer la qualité du modèle

En apprentissage automatique, il est important de définir ce que nous cherchons. Pour une tâche de régression, le modèle doit prédire une valeur numérique. L'erreur commise est donc une "différence" entre la valeur prédite et la valeur que nous aurions du avoir, la valeur réelle.

Cette différence peut être mesurée

- en valeur absolue
- en valeur quadratique
- etc.

C'est au développeur de choisir, en fonction de la réalité de son application car chacune de ces "différence" a des propriétés spécifiques. Pour cet exemple, choisissons une différence quadratique. Nous allons mesurer l'erreur quadratique qui est la somme des différences au carré calculée sur l'ensemble des données ; elle s'écrit :

$$Erreur = \sum_i (Note_i - \widehat{Note_i})^2 \quad (14)$$

où $Note_i$ est la note réelle obtenue par le i^{eme} étudiant et $\widehat{Note_i}$ est la note prédite par notre modèle pour ce même étudiant.

En pratique, cette fonction d'erreur dépend des paramètres du modèle, dans notre cas : *Pente* et *Connaissance*, et du jeu de données : $(t_i, Note_i)_{i=1,2,3}$.

$$Erreur(Pente, Connaissance, (t_i, Note_i)_{i=1,2,3}) = \sum_i (Note_i - (Pente \times t_i + Connaissance))^2 \quad (15)$$

Note

En apprentissage automatique, l'erreur est une fonction des paramètres du modèle et du jeu de données d'apprentissage est appelée **fonction de coût** (*loss function*). Notre tâche consiste à **estimer les paramètres** de notre modèle afin de **minimiser la fonction de coût**.

3.1.4 Première simplification

Nous devons pour l'instant trouver la valeur de deux paramètres : la *Connaissance* et la *Pente*. Ces deux paramètres peuvent être obtenus simultanément par descente de gradient. Dans un premier temps, afin de simplifier l'explication de cette méthode, nous allons fixer la valeur de la *Pente* et ne chercher que la valeur de la *Connaissance*.

La valeur de la pente est fixée grâce. la méthode des moindres carrés :

$$Pente = \frac{3 \times (20 + 44 + 75) - 11 \times 36}{3 \times (4 + 16 + 25) - 11^2} = 1,5 \quad (16)$$

Notre modèle devient :

$$Note = 1,5 \times t + Connaissance \quad (17)$$

3.2 Descente de gradient

Dans la section précédente nous avons :

1. **défini notre problème** : prédire la note en fonction du temps de travail
2. **défini nos données d'apprentissage** : les données fournies par 3 étudiants (attention à la gestion de données personnelles)
3. **choisi un modèle** : fonction linéaire
4. **choisi une fonction de coût** : l'erreur quadratique
5. **fixé l'un des paramètres de ce modèle pour simplifier notre tâche** (la *Pente* est fixée).

Le modèle choisi est donc :

$$Note = 1,5 \times t + Connaissance \quad (18)$$

La fonction de coût :

$$Erreur(Connaissance, (t_i, Note_i)_{i=1,2,3}) = \sum_i (Note_i - (1,5 \times t_i + Connaissance))^2 \quad (19)$$

Et il nous faut maintenant déterminer la meilleure valeur du paramètre *Connaissance* qui minimise la fonction de coût afin d'être en mesure de prédire les notes pour n'importe quelle durée de révision. C'est là que nous utilisons la descente de gradient pour prédire la valeur optimale de *Connaissance* cf. Figure 29.

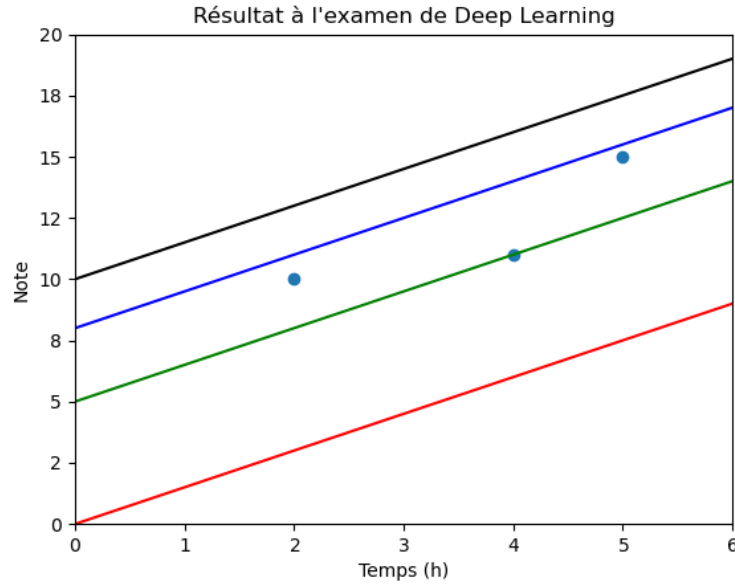


FIGURE 29 – La descente de gradient est utilisé dans cet exemple pour déterminer la meilleure ordonnée à l'origine pour notre modèle linéaire. Les différentes figures correspondent à différents modèles obtenus pour différentes valeurs du paramètre *Connaissance*.

3.2.1 Descente de gradient pour un paramètre

La descente de gradient est un algorithme itératif qui nécessite l'initialisation des paramètres à estimer.

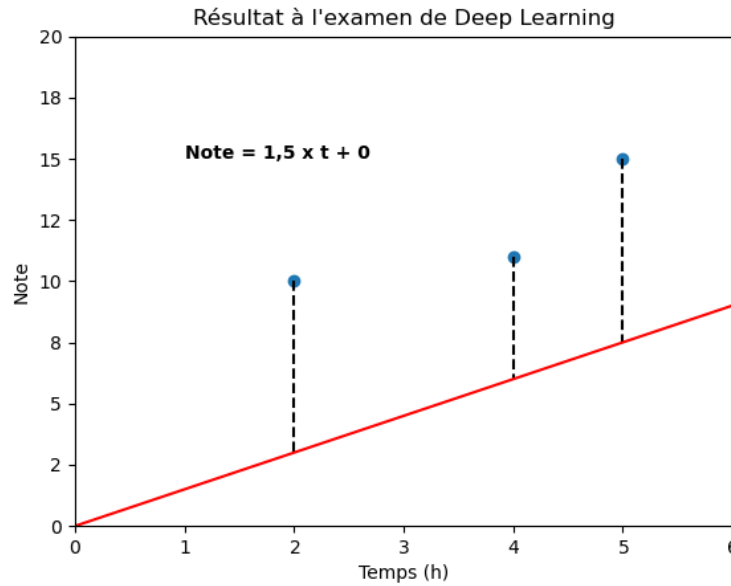
3.2.1.1 Initialisation Dans notre cas nous fixons aléatoirement la valeur du paramètre *Connaissance* à 0 (mais nous pourrions choisir n'importe quelle valeur).

Avec :

$$Pente = 1,5 \tag{20}$$

$$Connaissance = 0 \tag{21}$$

Le modèle obtenu est illustré par la figure 30. L'erreur quadratique obtenue est alors :

FIGURE 30 – Le modèle linéaire est initialisé avec *Connaissance* = 0.

$$Erreur(Connaissance, data) = (10 - 1,5 \times 2 + 0)^2 \quad (22)$$

$$+ (11 - 1,5 \times 4 + 0)^2 \quad (23)$$

$$+ (15 - 1,5 \times 5 + 0)^2 \quad (24)$$

$$Erreur(Connaissance, data) = 130,5 \quad (25)$$

L'algorithme de descente du gradient va maintenant permettre d'améliorer notre modèle (c-à-d la valeur du paramètre *Connaissance*) afin de réduire l'erreur obtenue sur nos données d'apprentissage.

3.2.1.2 Aperçu de la fonction de coût La figure 32 illustre quelques valeurs prises par la fonction de coût pour différentes valeurs du paramètre *Connaissance*. Idéalement, il faudrait pour voir estimer la fonction de coût pour toutes les valeurs de *Connaissance* afin de trouver le minimum. Une façon brutale d'optimiser ce paramètre consiste à calculer la fonction de coût pour un grand nombre de valeurs.

Note

Dans notre cas il est relativement simple de résoudre ce problème d'optimisation car la fonction de coût est très simple. Il suffit juste de dériver la fonction et de trouver son minimum en annulant la dérivée. Dans un cas réel, la fonction de coût dépend de milliers, millions, milliards de paramètres et ce n'est pas possible.

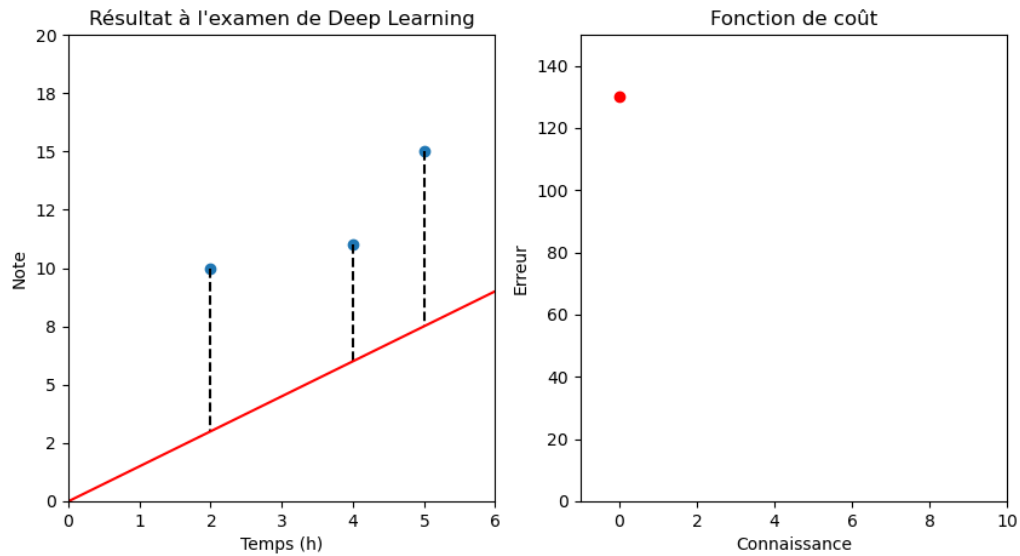


FIGURE 31 – Le modèle linéaire est initialisé avec $Connaissance = 0$. L'erreur quadratique est reportée sur le graphe de droite en fonction de la valeur du paramètre $Connaissance$.

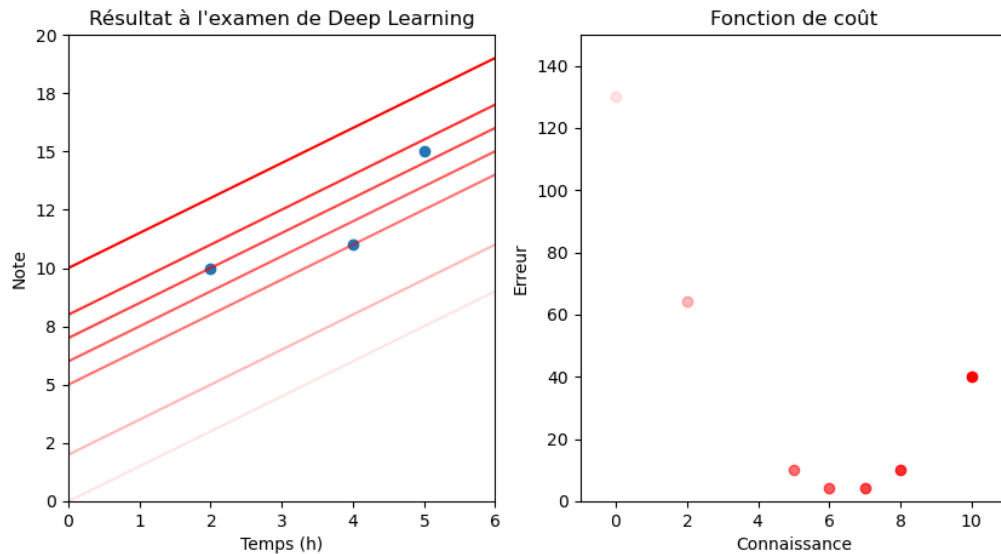


FIGURE 32 – Le modèle linéaire est initialisé avec $Connaissance = 0$. L'erreur quadratique est reportée sur le graphe de droite en fonction de la valeur du paramètre $Connaissance$.

3.2.1.3 Intérêt de la descente de gradient L'algorithme de descente de gradient présente plusieurs avantages.

- Il est utilisable pour un grand nombre de paramètres
- il converge vers un minimum de la fonction de coût (même si ce minimum peut être local)

et il augmente le nombre de valeurs de la fonction de coût calculé au voisinage de son minimum comme illustré par la figure 33.

- il calcule de plus en plus de valeurs lorsqu'il se rapproche de l'optimal : l'algorithme de descente de gradient optimise les paramètres en effectuant de grands pas lorsqu'il est loin de l'optimal et de petits pas lorsqu'il s'en rapproche.

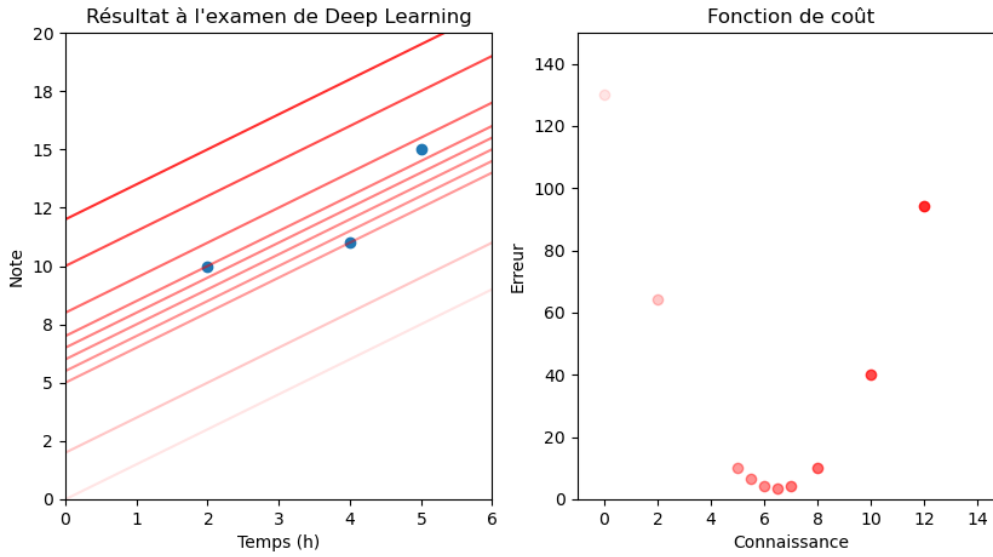


FIGURE 33 – L'algorithme de descente de gradient calcule de plus en plus de valeur lorsqu'il se rapproche de la solution optimale.

3.2.1.4 Dérivation Lors de l'initialisation de l'algorithme nous avons fixé la valeur initiale de *Connaissance* à zéro de manière arbitraire. Avec ce modèle, pour calculer l'erreur quadratique nous utilisons toutes les données d'apprentissage dans la formule suivante :

$$Erreur(Connaissance, (t_i, Note_i)_{i=1,2,3}) = \sum_i (Note_i - (1,5 \times t_i + Connaissance))^2 \quad (26)$$

Ce qui en pratique se traduit par :

$$Erreur(Connaissance) = (10 - (1,5 * 2 + Connaissance))^2 \quad (27)$$

$$+ (11 - (1,5 * 4 + Connaissance))^2 \quad (28)$$

$$+ (15 - (1,5 * 5 + Connaissance))^2 \quad (29)$$

$$(30)$$

Il est donc facile de calculer la valeur de l'erreur pour n'importe quelle valeur du paramètre

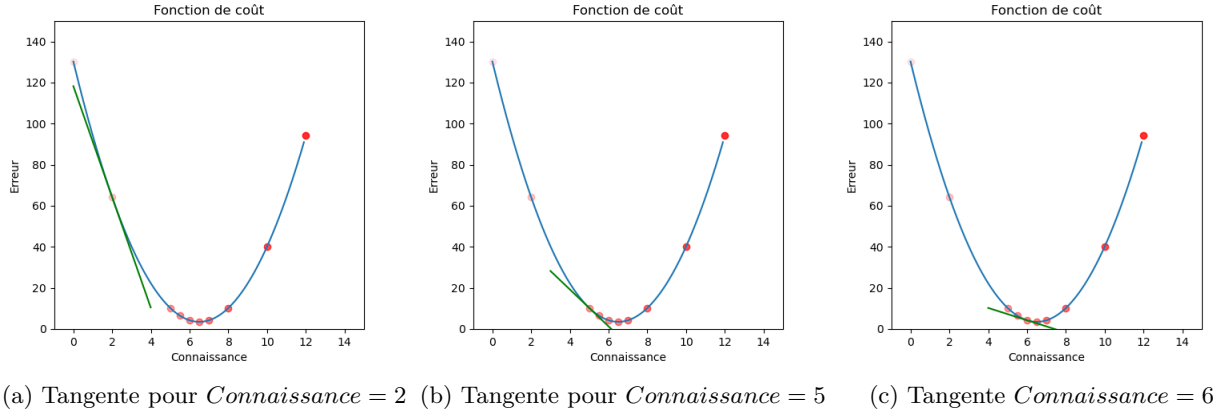


FIGURE 34 – Fonction de coût en fonction de la valeur du paramètre *Connaissance*. Chaque sous-figure montre la tangente à cette fonction en un point différent.

Connaissance et de calculer la dérivée de cette fonction en tout point. La figure 39 montre les tangentes à la courbe de la fonction de coût en différents points.

La dérivée de la fonction de coût par rapport à la *Connaissance* est :

$$\frac{dErreur(Connaissance)}{dConnaissance} = 2 \times (10 - (1,5 \times 2 + Connaissance))(-1) \quad (31)$$

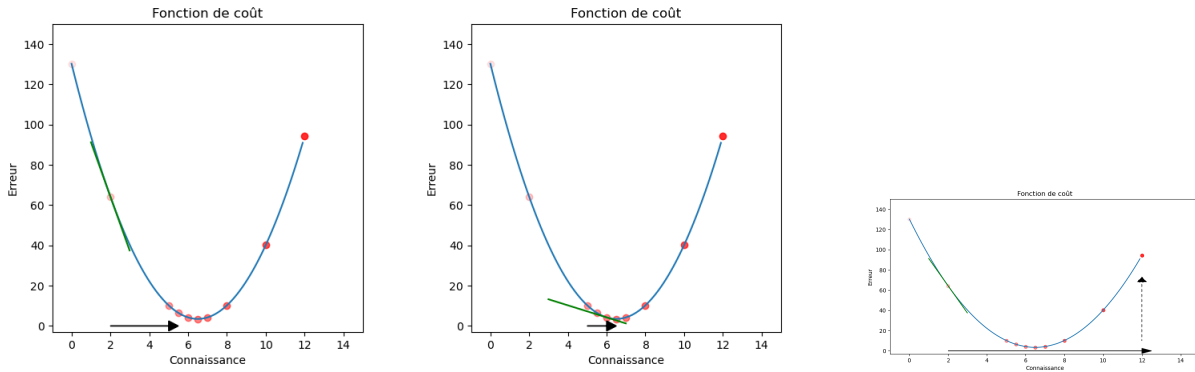
$$+ 2 \times (11 - (1,5 \times 4 + Connaissance))(-1) \quad (32)$$

$$+ 2 \times (15 - (1,5 \times 5 + Connaissance))(-1) \quad (33)$$

$$(34)$$

À l'initialisation du modèle, $Connaissance = 0$ et :

$$\frac{dErreur}{dConnaissance} = 6 \times Connaissance - 39 \quad (35)$$



(a) Tangente pour $Connaissance = 2$ (b) Tangente pour $Connaissance = 5$ (c) Tangente pour $Connaissance = 12$

FIGURE 35 – Fonction de coût en fonction de la valeur du paramètre $Connaissance$. Chaque sous-figure montre la tangente à cette fonction en un point différent et illustre la taille du pas nécessaire à la mise à jour du paramètre $Connaissance$. Si le pas est trop grand (figure de droite) la valeur de la loss ne décroît pas.

Note

Dans ce cas, simple, il suffit de déterminer le point pour lequel la pente de la tangente est nulle. La descente de gradient va, elle, permettre de converger vers ce point. C'est utile dans le cas où la fonction est trop complexe pour trouver le point où sa dérivée s'annule.

En observant les figures 39a, 39b et 34c, vous pouvez remarquer que la pente de la tangente à la courbe de la fonction de coût tend vers zéro lorsqu'on s'approche de son minimum. Pour $Connaissance = 2$, la pente est de -27, pour $Connaissance = 5$ elle est de -9 et pour $Connaissance = 6$ elle est de -3. Ainsi lorsque $Connaissance = 2$ il faut se déplacer d'une grande distance (modifier la valeur du paramètre $Connaissance$ d'un grand pas) mais lorsque $Connaissance = 5$ il faut se déplacer d'un pas plus petit comme illustré sur les figures 35a et 35b.

La descente de gradient détermine la taille du pas à effectuer pour mettre à jour le paramètre $Connaissance$ en multipliant la pente de la tangente au point actuel (dérivée de la fonction de coût au point actuel) par une valeur choisie par le développeur : le taux d'apprentissage ou **Learning rate**.

$$Stepsize = \frac{dloss}{dConnaissance} \times lr \quad (36)$$

La valeur du **learning rate** est généralement de l'ordre de 10^{-3} à 10^{-6} mais cela dépend de nombreux paramètres. Pour notre exemple prenons $lr = 0.05$.

Étape	Pas	Nouvelle valeur de Connaissance
0	1,95	$0 + 1,95 = 1,95$
1	1,365	$1,95 + 1,365 = 3,315$
2	0,9555	$3,315 + 0,9555 = 4,2705$
3	0,66885	$4,2705 + 0,66885 = 4,93935$
4	0,468195	$4,93935 + 0,468195 = 5,407545$
5	0,3277365	$5,407545 + 0,3277365 = 5,7352815$
6	0,229415549	$5,7352815 + 0,229415549 = 5,96469705$
7	0,160590	$5,96469705 + 0,160590 = 6,1252879$
8	0,11241363	$6,1252879 + 0,11241363 = 6,23770153$
9	0,078689	$6,23770153 + 0,078689 = 6,316391$
10	0,055082	$6,316391 + 0,055082 = 6,3714737$
11	0,0385578	$6,3714737 + 0,0385578 = 6,410031$

Lors de l'initialisation nous avons fixé $Connaissance = 0$ et en ce point la dérivée de la fonction de coût par rapport au paramètre $Connaissance$ est de -39, le pas est donc de

$$Stepsize = \frac{dloss}{dConnaissance} \times lr \quad (37)$$

$$= -39 \times 0.05 \quad (38)$$

$$= -1,95 \quad (39)$$

L'algorithme de descente de gradient nous dit que la nouvelle valeur du paramètre est donnée par :

$$Connaissance = Connaissance - \frac{dloss}{dConnaissance} \times lr \quad (40)$$

$$= 0 - (-39) \times 0.05 \quad (41)$$

$$= 1,95 \quad (42)$$

À la deuxième étape, avec $Connaissance = 1,95$ la dérivée de la fonction de coût est $-27,3$, la valeur du pas est donc $1,365$ et la nouvelle valeur de $Connaissance = 3,315$.

La figure 36 illustre les coûts successifs obtenus après 11 itérations de descente de gradient.

3.2.1.5 Quand arrêter ? La descente de gradient est un algorithme itératif. Comme la taille des pas diminue avec les itérations, il est possible de continuer sans fin à moins de fixer un **critère d'arrêt**. Il existe de nombreux critères d'arrêt dont les plus simples sont :

Nombre d'itérations maximum il suffit de fixer un nombre d'étapes à ne pas dépasser

Fixer un pas minimum une fois que le pas devient plus petit qu'un certain seuil, la mise à jour est insignifiante et on arrête l'algorithme

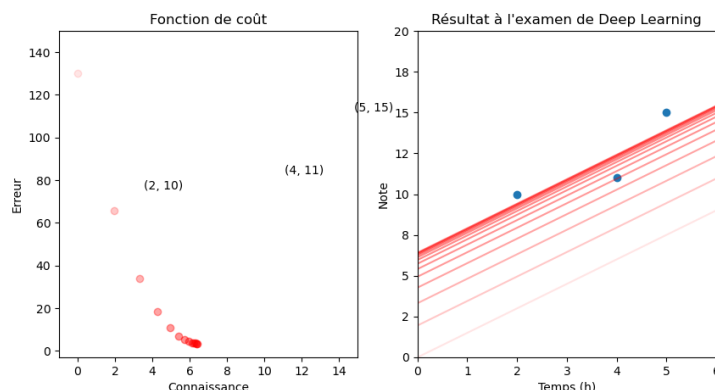


FIGURE 36 – Valeurs obtenus par la méthode de descente de gradient. La figure de droite montre l'évolution de la fonction de coût au cours des itérations et la figure de droite montre les droites obtenues pour chacune des itérations.

Souvent les deux critères sont utilisés conjointement, ce qui évite de faire des calculs inutiles. Il existe de nombreux algorithmes qui permettent d'améliorer l'algorithme de descente de gradient.

3.2.1.6 Résumé La méthode de descente de gradient s'articule en 8 étapes :

1. Choisir un modèle
2. Choisir une fonction de coût
3. Initialiser le paramètre
4. Dériver la fonction de loss par rapport au paramètre à estimer
5. Calculer la dérivé pour le modèle actuel
6. Utiliser cette dérivée pour calculer le pas de mise à jour
7. Mettre à jour la valeur du paramètre avec le pas
8. Boucler à l'étape 5 jusqu'à atteindre un critère d'arrêt

3.2.2 Descente de gradient pour plusieurs paramètres

L'exemple précédent permet de comprendre les différentes étapes de descente de gradient mais ne correspond pas à un cas réel. En pratique, les modèles disposent de nombreux paramètres. Voyons maintenant comment estimer plusieurs paramètres simultanément.

3.2.2.1 Modèle et fonction de coût Nous conservons un modèle linéaire mais en considérant maintenant que les deux paramètres : *Connaissance* et *Pente* sont inconnus. Le modèle

est donc :

$$Note = Pente \times t + Connaissance \quad (43)$$

Nous conservons la fonction de coût précédente mais elle devient une fonction de deux paramètres. En reprenant et adaptant l'équation 25

$$Erreur(Connaissance, Pente, data) = (10 - (Pente \times 2 + Connaissance))^2 \quad (44)$$

$$+ (11 - (Pente \times 4 + Connaissance))^2 \quad (45)$$

$$+ (15 - (Pente \times 5 + Connaissance))^2 \quad (46)$$

$$(47)$$

La fonction de coût en fonction des deux paramètres du modèle est représentée sur la figure 37

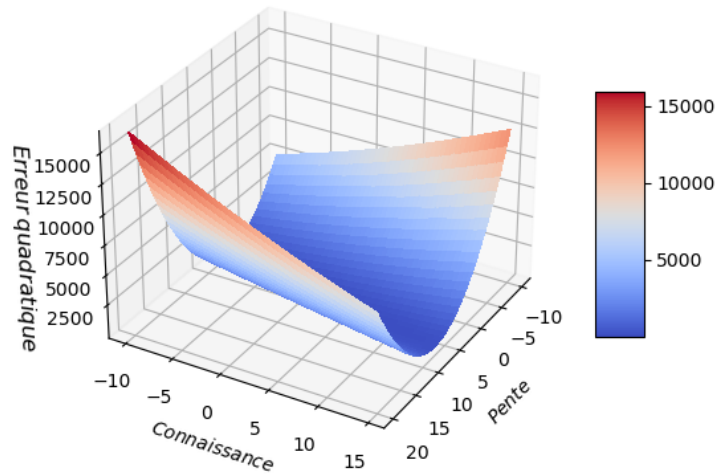


FIGURE 37 – La fonction de coût, dans cet exemple l'erreur quadratique, dépend de 2 paramètres, la *Pente* et la *Connaissance*.

3.2.2.2 Dérivation La fonction de coût dépend maintenant de deux paramètres et nous pouvons donc la dériver par rapport à la *Connaissance*, comme précédemment, ou par rapport

à la *Pente*. La dérivées partielle de la fonction de coût par rapport à la *Connaissance* est :

$$\frac{\partial loss}{\partial Connaissance} = \frac{\partial}{\partial Connaissance} (10 - (Pente \times 2 + Connaissance))^2 \quad (48)$$

$$+ \frac{\partial}{\partial Connaissance} (11 - (Pente \times 4 + Connaissance))^2 \quad (49)$$

$$+ \frac{\partial}{\partial Connaissance} (15 - (Pente \times 5 + Connaissance))^2 \quad (50)$$

$$\frac{\partial loss}{\partial Connaissance} = 2 \times (10 - (Pente \times 2 + Connaissance)) \times (-1) \quad (51)$$

$$+ 2 \times (11 - (Pente \times 4 + Connaissance)) \times (-1) \quad (52)$$

$$+ 2 \times (15 - (Pente \times 5 + Connaissance)) \times (-1) \quad (53)$$

$$\frac{\partial loss}{\partial Connaissance} = -2 \times (10 - 2Pente - Connaissance) \quad (54)$$

$$- 2 \times (11 - 4Pente - Connaissance) \quad (55)$$

$$- 2 \times (15 - 5Pente - Connaissance) \quad (56)$$

$$\frac{\partial loss}{\partial Connaissance} = 22 \times Pente + 6 \times Connaissance - 72 \quad (57)$$

Et la dérivée de la fonction de coût par rapport à la *Pente* est :

$$\frac{\partial loss}{\partial Pente} = 90 \times Pente + 22 \times Connaissance - 278 \quad (58)$$

Note

Lorsqu'une fonction dépend de plusieurs variables, ses dérivées partielles sont appelées des **gradients**.

De la même façon que dans l'exemple précédent, nous allons utiliser les dérivées de la fonction de coût, c-à-d ses gradient pour atteindre le minimum de cette fonction.

3.2.2.3 Initialisation Comme précédemment nous initialisons notre modèle en choisissant :

$$\begin{cases} Pente &= 1 \\ Connaissance &= 0 \end{cases}$$

3.2.2.4 Itérations En utilisant le gradient de la fonction de coût et les paramètres initiaux du modèle nous pouvons calculer les gradients selon *Connaissance* et *Pente* :

$$\begin{cases} \frac{\partial \text{loss}}{\partial \text{Pente}}(0, 1) = -188 \\ \frac{\partial \text{loss}}{\partial \text{Connaissance}}(0, 1) = -50 \end{cases}$$

Ainsi nous pouvons calculer les nouvelles valeurs de *Pente* et *Connaissance* :

$$\begin{cases} \text{Pente} = \text{Pente} - \frac{\partial \text{loss}}{\partial \text{Pente}}(0, 1) * lr \\ \text{Connaissance} = \text{Connaissance} - \frac{\partial \text{loss}}{\partial \text{Connaissance}}(0, 1) * lr \end{cases}$$

Si on fixe $lr = 0.05$ le pas est trop grand, on peut réduire la valeur de lr à 0.01. Remarquez que l'algorithme est très sensible au learning rate.

$$\begin{cases} \text{Pente} = 1 - (-188) * 0.01 \\ \text{Connaissance} = 0 - (-50) * 0.01 \end{cases}$$

$$\begin{cases} \text{Pente} = 2.88 \\ \text{Connaissance} = 0.5 \end{cases}$$

Ces nouvelles valeurs sont utilisées pour une nouvelle étape. Après 5000 itérations le modèle converge comme le montre les courbes de la figure 38. Quelques couples de valeur des paramètres (*Pente*, *Connaissance*) sont reportés sur la figure ?? afin de montrer la convergence de l'algorithme.

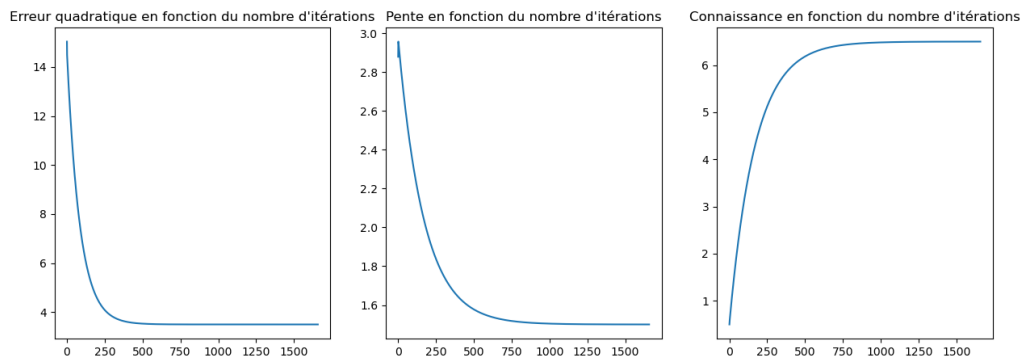
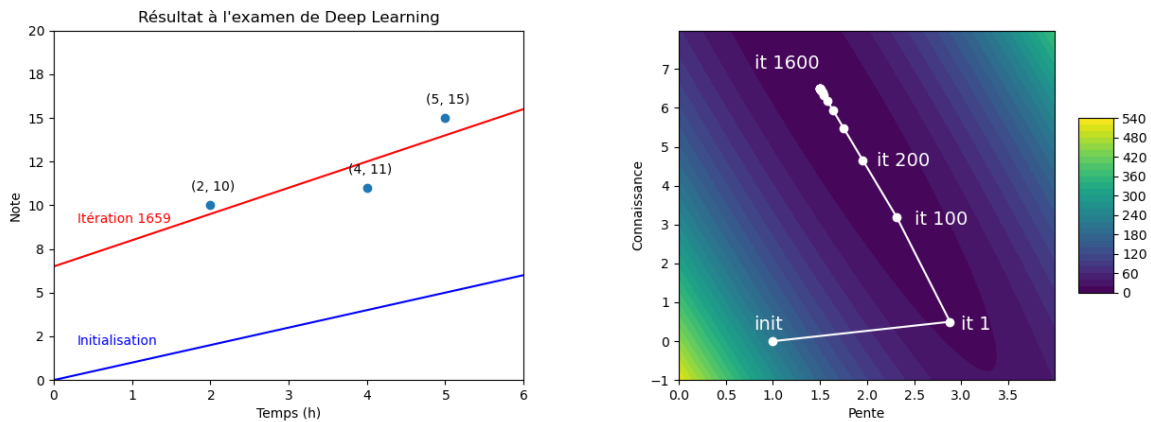


FIGURE 38 – Évolution de la fonction de coût et des deux paramètres *Pente* et *Connaissance* au cours des itérations

Le code Python correspondant à cet exemple est donné ci-dessous à titre d'exemple :

3 ENTRAÎNER UN RÉSEAU DE NEURONES PAR DESCENTE DE GRADIENT



(a) Modèle linéaire à l'initialisation et après 1659 itérations.

(b) Évolution des paramètres du modèle linéaire

FIGURE 39 – Résultat de la recherche de paramètres avec une descente de gradient.

```
# Training data
X = [2, 4, 5]
Y = [10, 11, 15]

# Set the learning rate
lr = 0.01

# Set the maximum number of steps
max_step = 5000

# Set a minimum update value
min_update = 0.00005

# Define the loss function
def loss_func(_p, _c, xs, ys):
    loss = 0
    for ii in range(3):
        loss += (ys[ii] - (_p * xs[ii] + _c))**2
    return loss

# Define the gradient of the loss function wrt Pente
def grad_pente(P, C):
    dp = 2 * ((Y[0] - X[0]*P - C) * (-X[0])
              + (Y[1] - X[1]*P - C)*(-X[1])
              + (Y[2] - X[2]*P - C)*(-X[2]))
    return dp

# Define the gradient of the loss function wrt Connaissance
def grad_connaissance(P, C):
    dc = 2 * ((Y[0] - X[0]*P - C) * (-1)
              + (Y[1] - X[1]*P - C)*(-1)
              + (Y[2] - X[2]*P - C)*(-1))
```

3.2.3 Remarques

3.2.3.1 A garder en tête

- Rappelez-vous que le choix du modèle est essentiel, il s'agit d'un méta-paramètre
- Le learning rate et la fonction de coût sont également des méta-paramètres qu'il vous faut fixer
- Le learning rate est sensible mais vous verrez par la suite qu'il existe de nombreux algorithmes pour trouver la bonne valeur
- Les critères d'arrêt sont également des méta-paramètres qui influent sur la qualité de votre modèle
- Il existe de nombreuses fonctions de coût qui peuvent être utilisées, cependant il est obligatoire que la fonction que vous utilisez soit dérivable.
- Dans notre exemple, la quantité de données d'apprentissage est très limitée (3 points) et le nombre de paramètres est limité à 2 théoriquement, la descente de gradient peut être utilisée pour des modèles beaucoup plus complexes et une quantité de donnée importante mais nous verrons dans la section suivante qu'il est nécessaire d'adapter cet algorithme dans ces cas précis.
- la descente de gradient nécessite que la fonction de coût soit continuellement dérivable.

3.2.3.2 Lien avec les réseaux de neurone Le lien avec les réseaux de neurones ne vous apparait peut être pas évident jusqu'ici mais une régression linéaire peut se mettre sous la forme illustrée par la figure 40 où l'on retrouve la structure d'un neurone artificiel sans fonction d'activation.

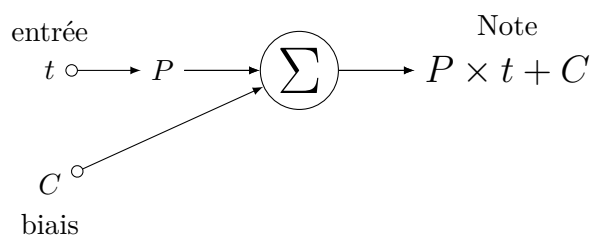


FIGURE 40 – Régression Linéaire mise sous forme d'un réseau de neurones linéaire.

4 Descente de gradient stochastique et mise en pratique

4.1 Descente de Gradient et Réseau de Neurones

Les réseaux de neurones sont organisés en couches successives afin de modéliser des fonctions complexes. Dans cette section nous allons voir comment estimer l'ensemble des paramètres d'un

réseau de neurones très simple grâce à une descente de gradient.

4.1.1 Données d'apprentissage

Nos données d'apprentissage consistent en un jeu de données *data* sont composées de deux échantillons $(0.5, 0.2)$, $(1.1, 1)$ pour lesquels les valeurs observées pour y sont respectivement 4 et 3.

4.1.2 Un réseau de neurones simple

La figure 40 montre la structure d'un réseau de neurones linéaire à une couche cachée. Nous ne traiterons pas des fonctions d'activation dans cette section pour simplifier les calculs.

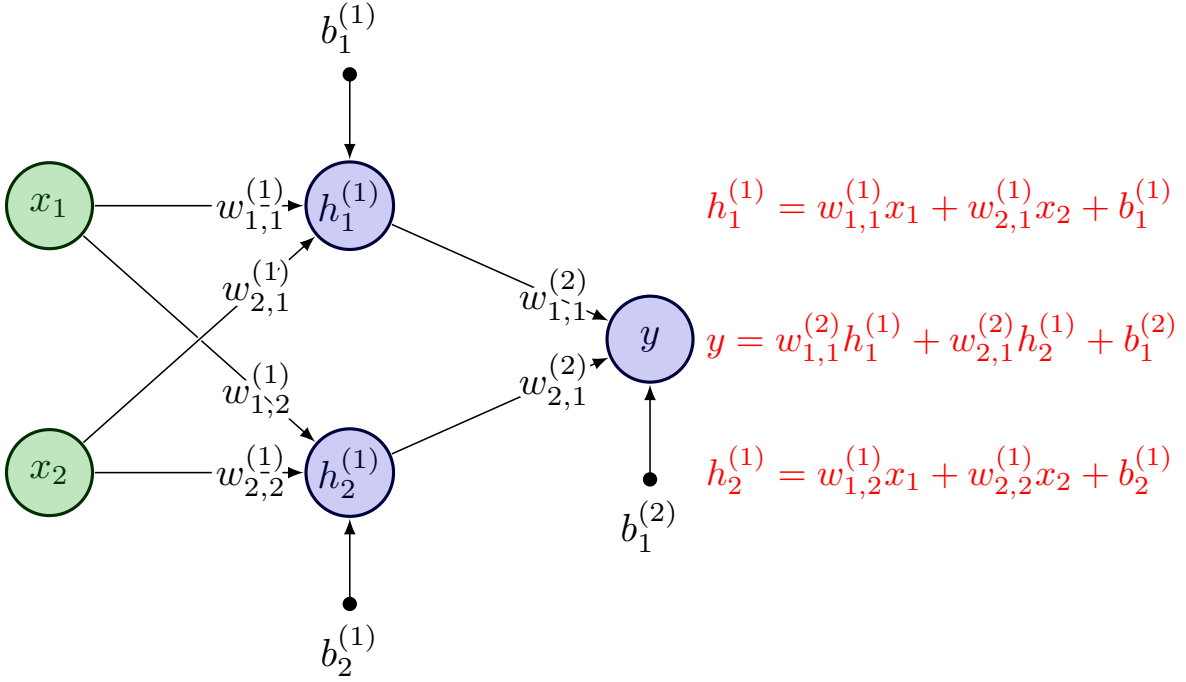


FIGURE 41 – Réseau de neurone linéaire à une couche cachée. Les deux entrées x_1 et x_2 sont pondérées et ajoutées aux biais dans chacun des neurones de la couche cachée. Les deux sorties, $h_1^{(1)}$ et $h_2^{(1)}$ des neurones de la couche cachée sont pondérées et ajoutés au biais de la couche de sortie. Aucune fonction d'activation n'est utilisée dans ce modèle pour simplifier les calculs.

4.1.2.1 Fonction de coût Comme précédemment nous utiliserons comme fonction de coût l'erreur quadratique.

$$Erreur(data, \Theta) = \sum_{d \in data} (y - \hat{y}(d))^2 \quad (59)$$

4.1.3 Descente de gradient et backpropagation

4.1.3.1 Initialisation Le modèle est initialisé avec :

$$w_{1,1}^{(1)} = 0.1$$

$$w_{1,2}^{(1)} = 0.2$$

$$w_{2,1}^{(1)} = 0.3$$

$$w_{2,2}^{(1)} = 0.4$$

$$w_{1,1}^{(2)} = 0.5$$

$$w_{2,1}^{(2)} = 0.6$$

$$b_1^{(1)} = 1$$

$$b_2^{(1)} = 1$$

$$b_1^{(2)} = 1$$

4.1.3.2 Forward pass Dans un premier temps nous calculons la sortie, y du réseau ainsi que les valeurs intermédiaires $h_1^{(1)}$ et $h_2^{(1)}$.

$$h_1^{(1)} = w_{1,1}^{(1)} \times x_1 + w_{2,1}^{(1)} \times x_2 + b_1^{(1)} \quad (60)$$

$$h_2^{(1)} = w_{1,2}^{(1)} \times x_1 + w_{2,2}^{(1)} \times x_2 + b_2^{(1)} \quad (61)$$

$$\hat{y} = w_{1,1}^{(2)} h_1^{(1)} + w_{2,1}^{(2)} h_2^{(1)} + b_1^{(2)} \quad (62)$$

En utilisant les données d'apprentissage on obtient :

$$h_1^{(1)}(0.5, 0.2) = 1.11 \quad (63)$$

$$h_1^{(1)}(1.1, 1) = 1.41 \quad (64)$$

$$h_2^{(1)}(0.5, 0.2) = 1.18 \quad (65)$$

$$h_2^{(1)}(1.1, 1) = 1.62 \quad (66)$$

4.1.3.3 Dérivation de la fonction de coût Nous avons vu précédemment qu'il faut, pour calculer le gradient, dériver la fonction de coût par rapport à chacun des paramètres du modèle. Nous avons ici 9 paramètres mais si l'on regarde de plus près il est possible de distinguer des paramètres de la couche de sortie, qui influent directement sur la valeur de $\hat{y}$ et donc sur l'erreur

(cf. equation 62) et les paramètres de la couche cachée qui influent sur les sorties des neurones de la couche cachée (equations 60 et 61 et de manière indirecte sur la valeur de l'erreur.

$w_{1,1}^{(2)}$ est un paramètre de la couche de sortie qui agit directement sur l'erreur.

$$\frac{\partial \text{Erreur}}{\partial w_{1,1}^{(2)}} = \frac{\partial}{\partial w_{1,1}^{(2)}} \left[y - (w_{1,1}^{(2)} h_1^{(1)} + w_{2,1}^{(2)} h_2^{(1)} + b_1^{(2)}) \right]^2 \quad (67)$$

$$= 2 \left[y - (w_{1,1}^{(2)} h_1^{(1)} + w_{2,1}^{(2)} h_2^{(1)} + b_1^{(2)}) \right] (-h_1^{(1)}) \quad (68)$$

De la même façon :

$$\frac{\partial \text{Erreur}}{\partial w_{2,1}^{(2)}} = 2 \left[y - (w_{1,1}^{(2)} h_1^{(1)} + w_{2,1}^{(2)} h_2^{(1)} + b_1^{(2)}) \right] (-h_2^{(1)}) \quad (69)$$

et

$$\frac{\partial \text{Erreur}}{\partial b_1^{(2)}} = 2 \left[y - (w_{1,1}^{(2)} h_1^{(1)} + w_{2,1}^{(2)} h_2^{(1)} + b_1^{(2)}) \right] \quad (70)$$

Les paramètres $w(1)_{1,1}$, $w(1)_{2,1}$, et $b_1^{(1)}$ appartiennent à la couche cachée et à ce titre ils influent indirectement sur la sortie $\hat{y}$ via la variable $h_1^{(1)}$. On peut donc utiliser l'équation 71 :

$$\frac{\partial \text{Erreur}}{\partial w_{1,1}^{(1)}} = \frac{\partial \text{Erreur}}{\partial h_1^{(1)}} \times \frac{\partial h_1^{(1)}}{\partial w_{1,1}^{(1)}} \quad (71)$$

Ainsi on peut calculer séparément les deux termes de l'équation 71.

$$\frac{\partial \text{Erreur}}{\partial h_1^{(1)}} = 2 \left[y - (w_{1,1}^{(2)} h_1^{(1)} + w_{2,1}^{(2)} h_2^{(1)} + b_1^{(2)}) \right] (-w_{1,1}^{(2)}) \quad (72)$$

$$\frac{\partial h_1^{(1)}}{\partial w_{1,1}^{(1)}} = x_1 \quad (73)$$

et ainsi :

$$\frac{\partial \text{Erreur}}{\partial w_{1,1}^{(1)}} = 2x_1 \left[y - (w_{1,1}^{(2)} h_1^{(1)} + w_{2,1}^{(2)} h_2^{(1)} + b_1^{(2)}) \right] (-w_{1,1}^{(2)}) \quad (74)$$

De la même façon les dérivées partielles par rapport aux autres paramètres sont calculés et le résultat est donné ci-dessous.

$$\frac{\partial \text{Erreur}}{\partial w_{2,1}^{(1)}} = 2x_2 \left[y - (w_{1,1}^{(2)} h_1^{(1)} + w_{2,1}^{(2)} h_2^{(1)} + b_1^{(2)}) \right] (-w_{1,1}^{(2)}) \quad (75)$$

$$\frac{\partial \text{Erreur}}{\partial b_1^{(1)}} = 2 \left[y - (w_{1,1}^{(2)} h_1^{(1)} + w_{2,1}^{(2)} h_2^{(1)} + b_1^{(2)}) \right] (-w_{1,1}^{(2)}) \quad (76)$$

et

$$\frac{\partial \text{Erreur}}{\partial w_{1,2}^{(1)}} = 2x_1 \left[y - (w_{1,1}^{(2)} h_1^{(1)} + w_{2,1}^{(2)} h_2^{(1)} + b_1^{(2)}) \right] (-w_{2,1}^{(2)}) \quad (77)$$

$$\frac{\partial \text{Erreur}}{\partial w_{2,2}^{(1)}} = 2x_2 \left[y - (w_{1,1}^{(2)} h_1^{(1)} + w_{2,1}^{(2)} h_2^{(1)} + b_1^{(2)}) \right] (-w_{2,1}^{(2)}) \quad (78)$$

$$\frac{\partial \text{Erreur}}{\partial b_2^{(1)}} = 2 \left[y - (w_{1,1}^{(2)} h_1^{(1)} + w_{2,1}^{(2)} h_2^{(1)} + b_1^{(2)}) \right] (-w_{2,1}^{(2)}) \quad (79)$$

Remarque

L'ensemble des équations qui permettent de calculer les dérivées partielles (le gradient) de la fonction de coût peut être mis sous forme d'un produit matriciel, ce qui facilite le calcul de l'étape suivante sur GPU.

4.1.3.4 Mise à jour des paramètres Nous allons ci-dessous calculer à la main la mise à jour des paramètres du modèle en utilisant le gradient de la fonction de coût et les données d'apprentissage. Comme dans l'exemple précédent (régression linéaire) nous allons utiliser l'ensemble des données d'apprentissage pour calculer par exemple pour le premier paramètre :

Note

La mise à jour des paramètres est effectuée en calculant la dérivée des sommes des erreurs (par exemple dans l'équation 81. Il est possible d'utiliser la moyenne des erreurs ; ce choix influera entre autres sur la valeur du learning rate.

$$\sum_{data} \frac{\partial \text{Erreur}}{\partial w_{1,1}^{(1)}}((x_1, x_2,), y) = \frac{\partial \text{Erreur}}{\partial w_{1,1}^{(1)}}((0, 5, 0.2), 4) \quad (80)$$

$$+ \frac{\partial \text{Erreur}}{\partial w_{1,1}^{(1)}}((1.1, 1), 3) \quad (81)$$

Ainsi en utilisant les données d'apprentissage pour la première étape on calcule :

$$\sum_{data} \frac{\partial Erreur}{\partial w_{1,1}^{(1)}} = -1.6792 \quad (82)$$

$$\sum_{data} \frac{\partial Erreur}{\partial w_{2,1}^{(1)}} = -2.01504 \quad (83)$$

$$\sum_{data} \frac{\partial Erreur}{\partial b_1^{(1)}} = -1.0844 \quad (84)$$

$$\sum_{data} \frac{\partial Erreur}{\partial w_{1,2}^{(1)}} = -1.30128 \quad (85)$$

$$\sum_{data} \frac{\partial Erreur}{\partial w_{2,2}^{(1)}} = -5.49228 \quad (86)$$

$$\sum_{data} \frac{\partial Erreur}{\partial b_2^{(1)}} = -5.83864 \quad (87)$$

$$\sum_{data} \frac{\partial Erreur}{\partial w_{1,1}^{(2)}} = -2.474 \quad (88)$$

$$\sum_{data} \frac{\partial Erreur}{\partial w_{2,1}^{(2)}} = -2.9688 \quad (89)$$

$$\sum_{data} \frac{\partial Erreur}{\partial b_1^{(2)}} = 4.948 \quad (90)$$

ces valeurs sont utilisées de la même manière que dans l'exemple précédent pour mettre à jour les paramètres. Par exemple pour le premier paramètre :

$$w_{1,1}^{(1)} = w_{1,1}^{(1)} - \frac{\partial Erreur}{\partial w_{1,1}^{(1)}} * lr \quad (91)$$

L'opération est répétée jusqu'à atteindre un critère d'arrêt. Le code Python correspondant est donné ci-dessous et disponible dans un notebook

4.2 Définition et principe

L'algorithme de descente de gradient vu dans la partie précédente permet d'apprendre les paramètres de modèles complexes en utilisant un jeu de données d'apprentissage. Nous avons vu que la mise à jour des paramètres est calculée à chaque itération en calculant le gradient de la fonction de coût sur l'ensemble des données d'apprentissage.

Dans nos exemples, pour une raison pédagogique, nous utilisons de très petits jeux de données mais dans un cas concret, la quantité de données nécessaire pour l'apprentissage peut compter des millions d'exemples de données en grande dimension. Ceci ajouté à la complexité

des modèles, c'est-à-dire au grand nombre de leurs paramètres à estimer pose d'importants problèmes d'échelle.

Dans l'équation 34 nous avons vu pour chaque paramètre du modèle, qu'en utilisant 3 observations pour l'apprentissage, le gradient de la fonction de coût était composé de 3 termes. Pour de nombreuses applications, le nombre de paramètre dépasse 100 millions et le nombre d'observations dépasse le million. Si on ajoute à cela qu'il est nécessaire de réaliser plusieurs centaines, voir milliers d'itérations avant que l'algorithme ne converge, on doit calculer au final plus de 10^{15} termes, ce qui pose des problèmes de temps de calcul mais également des problèmes numériques.

La descente de gradient stochastique permet de résoudre ce problème. Au lieu de mettre à jour les paramètres du modèle en utilisant l'ensemble des données d'apprentissage, chaque itération est effectuée en sélectionnant une petit ensemble de données d'apprentissage : un **batch**. À chaque itération, la descente de gradient stochastique sélectionne aléatoirement un petit nombre d'observations pour estimer le gradient et mettre à jour les paramètres. Cette méthode est très utile dans le cas où les données sont redondantes.

Reprenons l'exemple des parties précédentes et supposons que notre jeu de données soit composé des observations de 3 groupe d'étudiants ayant travaillé environ 2 heures, 4 heures et 5 heures. Les données d'apprentissage sont visibles sur la figure 42.

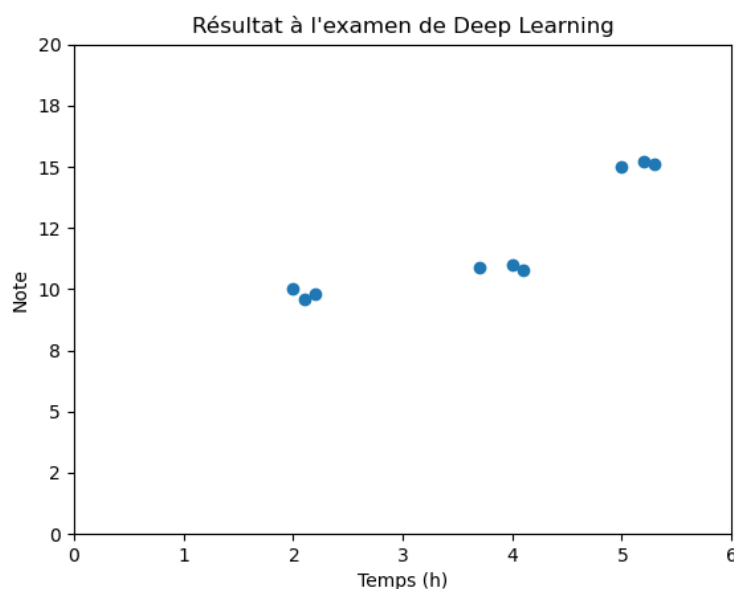


FIGURE 42 – Données d'apprentissage collectées auprès de 3 groupes d'étudiants.

Nous choisissons dans cet exemple d'utiliser des lots de 3 observations à chaque itération

(batchsize = 3). Ce choix est illustré sur la figure 43. Les 9 observations sont sélectionnées 3 par 3 pour effectuer une itération de l'algorithme de descente de gradient.

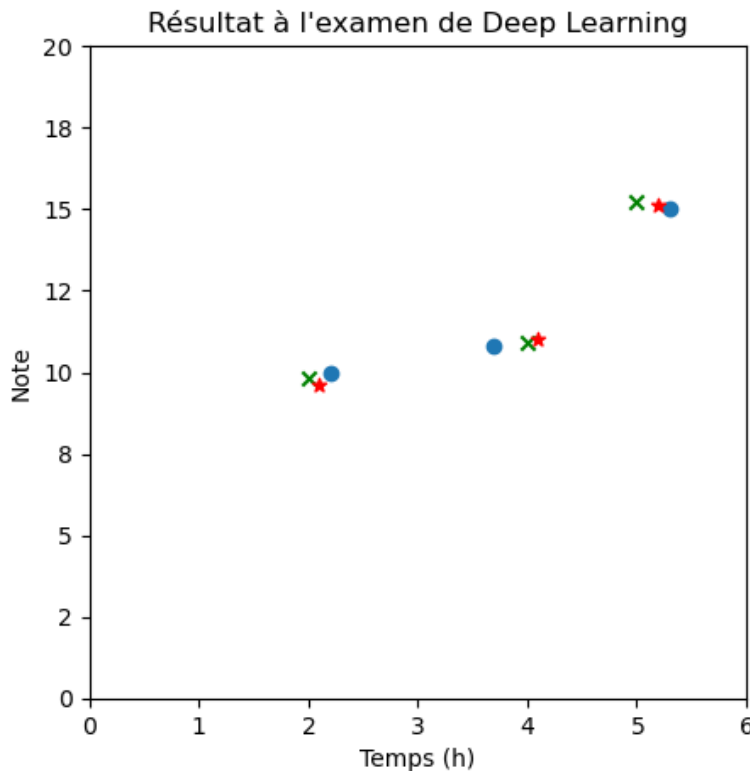


FIGURE 43 – Lots de données sélectionnés aléatoirement lors de la descente de gradient stochastique, chaque couleur représente un élément sélectionné à une itération différente..

En ne sélectionnant qu'un petit lot d'observations à chaque mise à jour du modèle, le coût de calcul se trouve grandement réduit mais rien de dit que l'algorithme de descente de gradient va converger. En pratique, dans des cas réels et avec une quantité de données d'apprentissage suffisantes, ces données sont fortement corrélées et les mises à jour du modèle mènent à des améliorations.

Du fait de l'utilisation d'une faible quantité de données à chaque itération, ce algorithme, appelé descente de gradient stochastique (**Stochastic Gradient Descent** ou SGD) n'est plus un algorithme de descente mais un algorithme qui descend "en moyenne". La fonction de coût ne diminue donc pas systématiquement à chaque itération.

Les batches de données sont sélectionnés aléatoirement sans remise dans le jeu de données d'apprentissage. Une fois que toutes les données ont été utilisées, on dit qu'on a réalisé une **époque**. On réitère alors le processus jusqu'à atteindre un critère d'arrêt.

La mise à jour du modèle est faite en sélectionnant, à chaque itération, un petit lot de

données appelé **batch**. Pour augmenter l'efficacité de cette méthode et réduire la quantité de calcul, la taille de ces batch doit être beaucoup plus petite que la taille du jeu de données. On appelle donc ces lots des **mini-batches**.

4.3 SGD en pratique

Intuitivement, on sent que si le gradient calculé pour chaque mini-batch varie trop, l'algorithme convergera difficilement car l'état du modèle va osciller à chaque mise à jour de ses paramètres. La variance des gradients est donc un élément important pour l'efficacité de cette méthode. Différentes méthodes permettent de réduire cette variance. Elles peuvent être classées en deux familles :

- les méthodes qui augmentent la taille de l'échantillon
- les méthodes qui utilisent un historique

L'optimisation de paramètres pour l'apprentissage automatique est un domaine de recherche à part entière, complexe et évoluant très rapidement. Dans cette section nous allons aborder certains concepts utiles à l'utilisation de la descente de gradient stochastique. Cette partie ne se veut pas exhaustive.

4.3.1 Variance et Batch size

Augmenter la taille des batch permet de réduire la variance entre les gradients successifs. Intuitivement, on voit que lorsque la taille des batches tend vers la taille du jeu de données, le gradient calculé pour un batch tend vers celui du jeu de données entier. On peut également, augmenter la taille des batches dynamiquement lors de l'apprentissage du modèle pour conserver un ratio vitesse/efficacité plus important. En pratique la taille du batch est généralement limitée par la capacité des machines et par la taille du modèle car l'ensemble des données (paramètres, gradients et données du batch) sont stockées dans la mémoire des cartes GPU.

Certaines méthodes comme *SVRG* (*Stochastic Variance Reduced Gradient*) ou *SAGA* (*Stochastic Averaged Gradient Algorithm*) reposent sur le principe d'agrégation du gradient pour calculer un gradient sur l'ensemble des données de façon régulière. Ces méthodes qui peuvent être efficaces dans certains cas sont très peu utilisées en apprentissage profond compte tenu de la taille des modèles et des jeux de données.

4.3.2 Momentum

Considérons qu'une étape du processus itératif de mise à jour du modèle s'écrit sous la forme :

$$x_{k+1} = x_k - \alpha g_k \tag{92}$$

où x_k est le paramètre à mettre à jour, α est le learning rate et g_k est une estimation du gradient calculé avec un seul échantillon de données ou un batch.

L'utilisation d'un **momentum** permet d'*accélérer* la mise à jour des paramètres du modèle.

$$x_{k+1} = x_k - \alpha(1 - \beta)g_k + \alpha\beta(x_k - x_{k-1}) \quad (93)$$

où β est un paramètre appelé **momentum** qui vérifie $\beta \in [0, 1]$. Dans l'équation 93, la mise à jour classique des paramètres en utilisant le gradient (terme $\alpha(1 - \beta)g_k$) est combiné avec la mise à jour précédente (terme $x_k - x_{k-1}$). Le momentum dote l'algorithme SGD d'une *mémoire* qui lui permet d'accélérer la mise à jour si la direction de mise à jour est la même que la précédente. Si la bonne direction est répétée, elle s'accélère alors que si la direction actuelle va à contresens de la précédente elles s'atténuent.

Ré-écrivons l'équation 93 pour voir apparaitre cet effet :

$$x_{k+1} = x_k - \alpha(1 - \beta)g_k + \alpha\beta(x_k - x_{k-1}) \quad (94)$$

$$= x_k - \alpha(1 - \beta)g_k + \alpha\beta(x_{k-1} - \alpha(1 - \beta)g_{k-1} + \alpha\beta(x_{k-1} - x_{k-2}) - x_{k-1}) \quad (95)$$

$$= x_k - \alpha(1 - \beta)g_k + \alpha\beta(-\alpha(1 - \beta)g_{k-1} + \alpha\beta(x_{k-1} - x_{k-2})) \quad (96)$$

$$= x_k - \alpha(1 - \beta)g_k + \alpha^2\beta(1 - \beta)g_{k-1} + \alpha^2\beta^2(x_{k-1} - x_{k-2}) \quad (97)$$

Le terme rouge dans l'équation 97 fait apparaitre l'accumulation des gradients.

4.3.3 Optimiseurs

4.3.3.1 AdaGrad

AdaGrad ou méthode du gradient adaptatif a été proposé pour aider à déterminer la valeur du learning rate. AdaGrad modifie chaque coordonnée du gradient (donc une valeur par paramètre du modèle) en fonction des valeurs précédentes du gradient pour cette coordonnée. Cette méthode estime une séquence $\{r_k\}_k$ telle que :

$$\forall i \in 1, \dots, d \begin{cases} [r_{-1}]_i &= 0 \\ [r_k]_i &= [r_{k-1}]_i + [g_k]_i^2 \forall k \leq 0 \end{cases} \quad (98)$$

ainsi une itération d'AdaGrad met à jour les paramètres selon l'équation suivante :

$$x_{k+1} = x_k - \alpha g_k \oslash \sqrt{r_k} \quad (99)$$

où $\oslash$ est l'opérateur de division par composante. Ainsi la vitesse de mise à jour de chaque paramètre évolue individuellement.

4.3.3.2 RMSProp

RMSProp pour *Root Mean Square Propagation* est un algorithme qui pondère le gradient à chaque itération dans le même esprit qu'AdaGrad. Comme AdaGrad, RMSProp calcule une séquence :

$$\forall i \in 1, \dots, d \begin{cases} [r_{-1}]_i &= 0 \\ [r_k]_i &= (1 - \lambda)[r_{k-1}]_i + \lambda[g_k]_i^2 \forall k \leq 0 \end{cases} \quad (100)$$

où $\lambda \in [0, 1]$. La valeur de λ permet de donner plus de poids au gradient actuel plutôt qu'à la valeur précédente de r_k .

4.3.3.3 Adam

Cette méthode combine le momentum vu précédemment et la pondération individuelle de chaque paramètre comme vu dans les méthodes AdaGrad et RMSProp.

La mise à jour des paramètres suit l'équation ?? :

$$x_{k+1} = x_k - \alpha m_k \oslash v_k \quad (101)$$

avec :

$$m_k = \frac{(1 - \beta_1) \sum_{j=0}^k \beta_1^{k-j} g_j}{1 - \beta_1^{k+1}} \quad (102)$$

et

$$v_k = \sqrt{\frac{(1 - \beta_2) \sum_{j=0}^k \beta_2^{k-j} g_j \odot g_j}{1 - \beta_2^{k+1}}} \quad (103)$$

où $\odot$ est le produit d'Hadamard (produit par composante).

v_k et m_k combinent des sommes pondérées des étapes précédentes et une pondération appliquée à quand dimension.

Adam est un des optimiseurs les plus populaires pour apprendre des réseaux de neurones profonds. D'autres versions de cet algorithme ont depuis été proposées comme **AdamW**.

